



OPEN

A unified framework for detecting point and collective anomalies in operating system logs via collaborative transformers

Mohammad Nasirzadeh, Jafar Tahmoresnezhad[✉] & Parviz Rashidi-Khazaei

Log anomaly detection is crucial for preserving the security of operating systems. Depending on the source of log data collection, various information is recorded in logs that can be considered log modalities. In light of this intuition, unimodal methods often struggle by ignoring the different modalities of log data. Meanwhile, multimodal methods fail to handle the interactions between these modalities. Applying multimodal sentiment analysis to log anomaly detection, we propose CoLog, a framework that collaboratively encodes logs utilizing various modalities. CoLog utilizes collaborative transformers and multi-head impressed attention to learn interactions among several modalities, ensuring comprehensive anomaly detection. To handle the heterogeneity caused by these interactions, CoLog incorporates a modality adaptation layer, which adapts the representations from different log modalities. This methodology enables CoLog to learn nuanced patterns and dependencies within the data, enhancing its anomaly detection capabilities. Extensive experiments demonstrate CoLog's superiority over existing state-of-the-art methods. Furthermore, in detecting both point and collective anomalies, CoLog achieves a mean precision of 99.63%, a mean recall of 99.59%, and a mean F1 score of 99.61% across seven benchmark datasets for log-based anomaly detection. The comprehensive detection capabilities of CoLog make it highly suitable for cybersecurity, system monitoring, and operational efficiency. CoLog represents a significant advancement in log anomaly detection, providing a sophisticated and effective solution to point and collective anomaly detection through a unified framework and a solution to the complex challenges automatic log data analysis poses. We also provide the implementation of CoLog at <https://github.com/NasirzadehMoh/CoLog>.

Keywords Log anomaly detection, Multimodal sentiment analysis, Point and collective anomaly detection, Class imbalance, Deep learning

Anything that happens in a system during its interactions is recorded in system logs, including time-stamped events (e.g., transactions, errors, and intrusions). Anomalies, also known as outliers, are patterns or events in log data that deviate from the system's usual behavior. Log-based anomaly detection identifies and locates log outliers¹. Nevertheless, manually analyzing logs or applying traditional methods^{2–19} becomes more challenging due to log data's large-scale, complicated, and dynamic nature, as these methods primarily rely on manual processing and rule-setting^{20–23}. Furthermore, log data originating from various systems may employ distinct terminology. Hence, deep learning-based automated log anomaly detection is crucial to address threats or challenges promptly, define reaction thresholds, and make data-driven decisions²⁴.

In recent years, deep learning has shown considerable promise in automating many tasks^{25–34}. One illustration of these tasks is the process of automated log anomaly detection. Due to deep learning techniques' outstanding results in automated log anomaly detection, utilizing these techniques for anomaly detection in log files can provide real-time alerts for crucial scenarios, including unidentified threats, identifying underlying causes, and mitigating cyberattacks, fraudulent activities, or system malfunctions^{35–39}. Consequently, deep neural networks have become the dominant modeling method in automated log-based anomaly detection in recent years. This research direction starts with DeepLog's revolutionary results in detecting anomalies from log files³⁶. Through the years, different deep learning-based approaches have evolved to become increasingly influential in this field⁴⁰. Numerous research directions exist within the automated log anomaly detection domain, such as multi-

Faculty of Information Technology and Computer Engineering, Urmia University of Technology, Band, Urmia 57166-17165, West Azerbaijan, Iran. ✉email: j.tahmores@it.uut.ac.ir

layer perceptrons (MLPs)^{41–45} that are used as part of other deep learning architectures, convolutional neural networks (CNNs)^{46–51}, recurrent neural networks (RNNs)^{36,38,52–67}, autoencoders^{42,68–72}, generative adversarial networks (GANs)^{42,52,73–78}, transformers^{21,79–85}, attention mechanisms^{37,41,48,86–89} that are usually employed to enhance performance in other deep neural architectures, graph neural networks (GNNs)⁹⁰, evolving granular neural networks (EGNNs)⁹¹, and large language models (LLMs)^{35,92,93}.

As demonstrated in Fig. 1, depending on the type of log data collecting source, a wide range of information can be stored in logs. Practically, this data is expressed through different aspects, which we refer these aspects to as modalities. Each log file follows two primary modalities: (1) the sequence of records, which we refer to as sequence modality, and (2) the content of each record, which we refer to as semantic modality.

On one side, the sequence of events feature can contribute information to each log event from a semantic perspective. This means that various types of information in a log file can be used to create a more accurate model. Figure 1 illustrates how sequence and semantic modalities are relevant and how respective sequence vectors can be utilized to distinguish identical events' vectors. On the other side, these modalities allow the model to make decisions from various perspectives, thereby enhancing the precision of outcomes when performing the assigned tasks.

On the one hand, most existing literature approaches concentrate on unimodal anomaly detection from logs^{21,35,37,38,41,43,46,47,49,50,52–54,56–58,60,63,64,68,70,71,82,87,89,90,92,93}. These studies only utilize a specific facet of the information recorded in the log. However, disregarding other modalities of a log leads to the forfeiture of valuable data. In fact, focusing the analysis of log files based on a singular aspect of their information can lead to the model's inability to identify more complex anomalies. On the other hand, the multimodal approaches proposed in the literature utilize early^{51,59,61,62,83,85,86,88,91}, intermediate^{42,45,48,55,65–67}, and late⁴⁴ fusion mechanisms or separate models^{36,69,84} for different modalities that they extract, leading to various challenges, which are presented in Table 1. Concatenating unprocessed data from several modalities can lead to very high-dimensional input, which can be challenging to handle due to high-dimensionality challenges. Noise in the raw data may affect the model because noise in one modality may influence the information as a whole, indicating noise sensitivity challenges. Concatenating raw data leads to complexity, particularly with diverse data types and formats, leading to heightened data complexity challenges. Features derived from several modalities may not consistently possess direct compatibility or effortless combinability, highlighting compatibility challenges. Deploying distinct models for each modality can augment the system's overall complexity, resulting in heightened network complexity challenges. The contributions of several modalities can be challenging, mainly when one modality provides more information than the others, causing balancing contribution challenges. Merging the outcomes at the decision level or utilizing distinct models for each modality might ignore significant interactions between modalities, resulting in a lack of shared information challenges. Performing processes separately for each modality can result in redundant computations, mainly if overlapping features exist, causing redundancy challenges. The

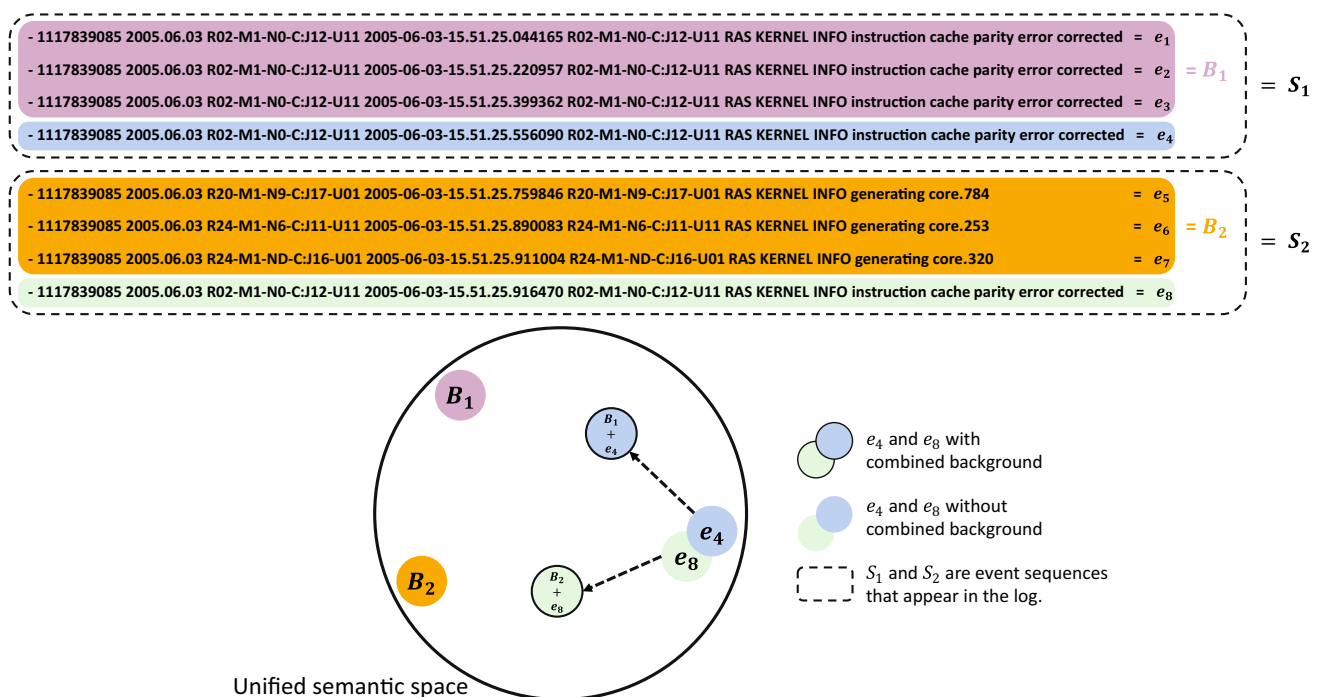


Fig. 1. Illustration of how the sequence modality (B_1, B_2) adds more information to semantic modality (e_4, e_8) and how the same samples with different backgrounds are separated in a unified semantic space. For the e_4 and e_8 event vectors, their backgrounds, specifically B_1 and B_2 , might be utilized to depict e_4 and e_8 in a more meaningful manner (it should be noted that e_4 is equivalent to e_8). In this sense, we designate the B_1 and B_2 as sequence modalities and the e_4 and e_8 as semantic modalities.

Challenges	Methods			
	Early Fusion	Intermediate Fusion	Late Fusion	Separate Models
High dimensionality	All except <i>MLog</i> ¹	<i>Mdfulog</i> ² , <i>Swisslog</i> ³ , <i>WDLLog</i> ⁴ , and <i>FSMFLog</i> ⁵	×	×
Noise sensitivity	All	×	×	×
Heightened data complexity	All	×	×	×
Compatibility of features	All	<i>Mdfulog</i> , <i>Swisslog</i> , <i>WDLLog</i> , and <i>FSMFLog</i>	×	×
Heightened network complexity	×	All	All	All
Balancing contributions	All	<i>Mdfulog</i> , <i>Swisslog</i> , <i>WDLLog</i> , and <i>FSMFLog</i>	All	×
Lack of shared information	×	×	All	All
Redundancy	×	×	All	All
Limited improvement	×	×	All	All

Table 1. Challenges arising from the multimodal approaches covered in the log anomaly detection literature. ¹Fu et al. proposed *MLog*. ²Li et al. proposed *Mdfulog*. ³Li et al. proposed *Swisslog*. ⁴Niu et al. proposed *WDLLog*. ⁵Niu et al. proposed *FSMFLog*.

potential for enhancing performance may be restricted because the modalities have not been merged until the final decision step, leading to limited improvement challenges.

Considering the above motivations, we propose a novel automated log-based anomaly detection approach - called CoLog - based on multimodal sentiment analysis (MSA)^{94–103} to overcome the above mentioned challenges. CoLog interprets anomalies in the log as negative sentiments. In the same way, normal samples are classified as positive sentiments. Since the two most important modalities of a log file are the semantic features of an event and the features obtained from the sequence of events. The background and context of an event can be formed as sequence modality. CoLog constructs semantic and sequence modalities to learn more about anomalies through the interaction between these modalities. During this process, we employ a form of the guided-attention (GA) mechanism¹⁰⁴ to encode each modality in an collaborative manner with other modalities. Finally, to accomplish the objective of prediction through integrating modalities, we apply transformer blocks and then transform the outcome feature vectors of each modality into a higher-dimensional space known as the latent space to perform meaningful fusion.

In addition, logs are often the place where the point and collective anomalies are recorded. Detecting point and collective anomalies in log records is crucial, as each anomaly type exposes distinct facets of system behavior. Point anomalies, representing isolated instances of abnormal behavior, often signal immediate issues like a sudden malfunction or attack. Collective anomalies are patterns of anomalous activity over time that may signify deeper, more systemic issues, such as steady performance degradation or persistent security threats. Detecting different kinds of abnormalities provides a thorough awareness of short-term and long-term issues, facilitating more effective troubleshooting and preventative care. However, all existing works solely concentrate on identifying only one type of log anomaly. Besides that, to the best of our knowledge, two techniques exist outside the log-based anomaly detection area that aim to identify point and collective anomalies utilizing a unified framework. Li et al. processed a time series using temporal convolutional network (TCN) and calculated the anomaly score to assess whether the point or collective data was anomalous based on the fixed-length sequences. However, due to its reliance on a predetermined sequence length, it may fail to identify certain instances of collective anomalies. Moreover, Liu et al. used an improved local outlier factor (ILOF) to detect variable-length collective anomalies where uses identified point anomalies as boundary points to partition the residual sequence into many subsequences of varied lengths. Each subsequence's anomaly score is computed using the ILOF method. This method relies on calculating anomaly scores, which can lead to false alarms, particularly in long sequences. Furthermore, neither of these approaches explicitly learns the relationships between the various modalities. Therefore, leveraging the transformer¹⁰⁷ architecture, we propose CoLog to handle the problem of detecting point and collective log anomalies by utilizing a unified framework. To achieve this objective, we propose utilizing collaborative transformer (CT) to process log modalities in an end-to-end manner, enabling them to interact and learn about their relationships. Since CoLog is a supervised approach, it does not face the challenge of accumulating errors.

The summarization of our contribution is as follows.

1. We propose CoLog, the first framework to formulate log anomaly detection as a multimodal sentiment analysis problem, enabling the detection of both point and collective anomalies in a unified manner.
2. We design a collaborative transformer architecture with impressed attention and a modality adaptation layer to capture nuanced interactions between semantic and sequence modalities.
3. We introduce a balancing mechanism to adaptively weight contributions of modalities, which helps to ensure that the modalities with inherent differences are represented in the same representation space.
4. We assess CoLog using seven benchmark datasets, achieving state-of-the-art performance, and provide the CoLog implementation to facilitate reproducibility in the GitHub repository.
5. Our research findings and specialized services are available on the [Alarmif](#) website, facilitating a direct link between research and practical application.

The paper is organized as follows. Sect. “Related work” discusses previous research in the domain. Define the task, the importance of transformers for this work, the threat model, and the assumptions outlined in Sect. “Preliminaries”. Section “Method” will discuss CT’s design and training considerations for anomaly detection from logs based on MSA. The evaluation is presented in Sect. “Experiments”, where we conclude our work and then offer suggestions for future research in Section “Conclusion”.

Related work

Numerous research endeavors have examined log-based anomaly detection^{1,20,22–24,40,108}. These approaches have evolved from regular expressions and rule-based methods to applying deep learning-based techniques. Historically, traditional techniques employed regular expressions¹⁹, rule-based methods^{5,6,8,9,15–18}, or machine learning (ML)^{4,12,109–111} to find and extract abnormal events. Regular expressions-based approaches often include formulating domain-specific regular expressions. These regular expressions are used to identify recently discovered vulnerabilities by comparing patterns. A primary drawback of these techniques is that they can overlook the latest anomalous events. Also, when each log entry appears normal, but the entire sequence is abnormal, no abnormal activity can be detected using regular expressions-based techniques. Rule-based methods often use predefined rules or signatures to detect abnormal patterns in log data. These principles are typically based on normal and abnormal system behavior observations. Data that conforms to these guidelines is classified as abnormal. The rule-based approaches can only recognize the predefined abnormalities. The ML-based log analysis process for anomaly detection comprises three primary steps: log parsing, feature extraction, and anomaly identification. Methods built on ML can not track the temporal information because they do not have a mechanism to remember past events. Besides, these approaches heavily impact the model output because they need human feature extraction from raw logs. In fact, all of the approaches mentioned above need extensive domain expertise. CoLog is a flexible approach that does not rely on domain expertise or predefined rules. CoLog exhibits a notable aptitude for extracting temporal information.

Outlier detection should be in real-time. Since deep neural networks are successful in real-time text analysis, numerous deep learning-based methods describe log data as a natural language sequence and effectively outlier finding¹⁰⁸. This started a trend of proposing advanced deep learning methods to identify log anomalies. These methods address the problems of approaches mentioned earlier.

Subsequent subsections outline the literature on deep learning for automated log-based anomaly detection. We then review the literature on deep MSA and compare our log anomaly detection approach to deep learning methods.

Deep learning for anomaly detection from logs

The inception of deep learning-based methodologies can be traced back to the advent of DeepLog³⁶, which demonstrated commendable efficiency in detecting log data anomalies. Due to their rapid evolution over the past few years, these techniques now serve as the standard for modeling in this domain. Hence, the present subsection is most centered on models based on deep learning. In the following, we discuss unimodal and multimodal approaches.

Unimodal approaches

This subsection is dedicated to unimodal models for log anomaly detection despite the existence of various developed deep learning techniques for this purpose. These techniques’ primary concept is derived from natural language processing (NLP). Furthermore, they frequently prioritize the sequential nature of log data. Deep learning different techniques, including MLPs^{41,43}, CNNs^{46,47,49,50}, RNNs^{38,52–54,56–58,60,63,64}, autoencoders^{68,70–72}, GANs^{52,73–78}, transformers^{21,79–82}, attention mechanisms^{37,41,87,89}, GNNs⁹⁰, and LLMs^{35,92,93}, are frequently employed in these approaches. Except⁶⁸, the unimodal approaches mentioned earlier can be divided into three main categories: sequence-based, semantic-based, and LLM-based. Otomo et al. proposed a log event anomaly detection method for large-scale networks that embeds diverse log data into hidden states using latent variables without requiring any preprocessing or feature extraction. The key idea is to first translate raw log messages into log time series for each log type, then map the log time series into latent variables per day and per log type using a conditional variational autoencoder. Finally, a clustering method is applied to the latent variables to detect deviations from the detected clusters, interpreted as anomalies.

Sequence-based unimodal approaches: Sequence-based modeling^{21,37,41,43,46,47,49,50,54,56,57,60,63,64,70–75,77–79,81,87,89,90} is widely recognized as the dominant network architecture in log-based anomaly detection. Typically, these methods interpret the log data as a sequence of natural language. For example, Lu et al. extracted log keys from raw logs and assigned them unique numerical values. Shorter vectors are padded with zeros, while longer vectors are truncated to ensure that each session of log keys has the same length. The logs in vectorized form are inputted into an embedding layer, followed by three 1D convolutional layers with varying filter sizes. After that, max-pooling, dropout, and a fully connected softmax layer are applied to classify anomalies. Zhang et al. demonstrated that log data is unstable due to the evolution of logging statements and noise in log data processing. They proposed LogRobust to tackle this issue by extracting semantic feature vectors from log event sequences. Then, an attention-based bidirectional long short-term Memory (BiLSTM) model is proposed to automatically identify abnormal instances and learn about the value of various log events. LogSpy⁴¹ is a method for detecting anomalies in distributed systems. It uses a technique called frequent template tree (FT-Tree) to extract templates and the skip-gram model to construct feature vectors. It employs a CNN architecture and an attention mechanism to analyze the relationships between log templates and improve the efficiency of real-time log anomaly detection. LogBERT⁷⁹ uses the transformer encoder to model log sequences for self-supervised log anomaly detection. The training process involves two distinct tasks: (1) masked log key prediction to accurately identify randomly masked log keys in normal sequences and (2)

volume of hypersphere minimization to bring normal log sequences close in the embedding space. LogBERT encodes normal log sequence patterns and detects anomalies after training. LogGAN⁷⁴ uses a log-level GAN with permutation event modeling. A log parser converts unstructured system logs into structured events. Permutation event modeling minimizes long short-term memory (LSTM)'s sequential dependency concerns, allowing it to handle out-of-order log arrivals. The adversarial learning system uses a generator to generate synthetic logs and a discriminator to verify them. DeepCASE⁸⁷ can acquire knowledge regarding the correlation among log events in sequences. Subsequently, an interpreter arranges similar events into similar clusters to detect abnormalities. The system administrator receives reports of abnormality samples, and the administrator's decisions are learned by DeepCASE and applied to future occurrences of identical sequences. Catillo et al. proposed AutoLog as a semi-supervised deep autoencoder. Its functionality is not contingent upon the concept of log line sequencing. The approach employed involves the extraction of numeric score vectors to handle heterogeneous logs. During the process, AutoLog does not incorporate any application-specific knowledge and refrains from making any presumptions regarding the format and order of the underlying lines within the logs. Qi et al. proposed a log-based anomaly detection approach known as Adanomaly. This method employs the bidirectional GAN model for extracting features and a technique for classification based on ensemble learning to detect anomalies. Adanomaly calculates the reconstruction loss and discriminative loss as features with the help of bidirectional GAN. LogEncoder⁴³ is a framework that uses labeled and unlabeled data to detect anomalies in system logs. The system comprises three primary elements: Log2Emb, Emb2Rep, and Anomaly Detection. The Log2Emb procedure transforms discrete log events into semantic vectors. The Emb2Rep procedure employs an attention-based LSTM model to differentiate between normal and abnormal sequences. The framework is adaptable for both offline and online detection. Zhang et al. proposed LayerLog, an innovative system for detecting anomalies in log sequences. LayerLog utilizes a hierarchical structure called "Word-Log-Log Sequence" to do this. The system comprises three layers: Word, Log, and LogSeq. It extracts semantic information from raw logs without any loss. Empirical assessments demonstrate the efficacy of LayerLog in detecting abnormalities. LogFormer³⁷ is a pipeline consisting of two stages: pre-training and tuning. It is designed to detect log anomalies and enhance generalization across different domains. It extracts semantic information of log sequences and trains a transformer-based model on a specific domain. By employing adapter-based tuning, the training costs are minimized by keeping most parameters fixed and only updating a small number of them. FastLogAD⁷⁷ is an unsupervised technique for detecting anomalies in log data. It utilizes a mask-guided anomaly generator (MGAG) and a discriminative abnormality separation (DAS) model. The MGAG algorithm produces simulated anomalies based on normal log data, while the DAS model distinguishes between normal log sequences and simulated ones, allowing for effective real-time detection. MLAD⁸¹ is a log anomaly detection approach that utilizes the transformer and gaussian mixture model (GMM) to detect anomalies across many systems. MLAD employs SBERT¹¹² to transform log sequences into semantic vectors. It utilizes a sparse self-attention mechanism to capture word-level dependencies and a GMM to emphasize the uncertainty associated with infrequent words in the "identical shortcut" problem. Zhang et al. proposed E-Log, which provides fine-grained elastic anomaly detection and diagnosis for databases. Qiu et al. introduced LogAnomEX, an unsupervised log anomaly detection method based on the Electra model and gated bilinear neural networks. Qiu et al. proposed FedAware, a distributed IoT intrusion detection method that leverages fractal shrinking autoencoders. These methods highlight both supervised and unsupervised perspectives, reinforcing the importance of comparative baselines.

Semantic-based unimodal approaches: Semantic-based unimodal log anomaly detection is limitedly used^{38,52,53,58,76,80,82}. Several semantic-based approaches utilize log lines to extract semantic information and build models using unsupervised or semi-supervised approaches^{76,80,82}. Other techniques incorporate sentiment analysis (SA) to detect log anomalies^{38,52,53,58}. These methods utilize the polarity of log messages as a basis for learning and predicting normal and abnormal log events. SA-based log anomaly detection methods classify normal occurrences as positive sentiments and anomalies as negative. For example, Farzad and Gulliver proposed an approach for detecting and classifying anomalies in log messages by employing three different deep learning architectures: Auto-LSTM, Auto-BiLSTM, and Auto-gated recurrent unit (GRU). The methodology consists of two primary phases: feature extraction with autoencoders separately for positive and negative classes and concatenating outputs to provide a single-label dataset for training an LSTM, BiLSTM, and GRU model. The classification model is trained using the softmax activation function, categorical cross-entropy loss, and the Adam optimizer¹¹⁶. Farzad also proposed a log message anomaly detection method using a combination of GAN, autoencoder, and GRU networks. It generates synthetic negative log messages using a proposed SeqGAN to oversample negative log messages and balance the dataset. The autoencoder extracts features from positive and oversampled negative log messages, with separate networks for each class. A GRU network is used for anomaly detection and classification, with improved accuracy compared to models without over-sampling. Logsy⁸⁰ is a classification-based method for detecting anomalies in unstructured log data. It uses a self-attentive encoder and a hyperspherical loss function to learn compact log vector representations. It uses auxiliary log datasets from other systems to improve normal log representation and prevent overfitting. The learned representations help distinguish normal and abnormal logs using the anomaly score as a distance from the center of a hypersphere. Studiawan et al. proposed pylogsentiment, a SA-based method for identifying anomalies in production OS log files. This approach uses positive and negative sentiment analysis, with negative sentiment detection similar to detecting anomalies. They used deep learning to predict unseen data and implemented a GRU network to determine sentiments. They considered class imbalance due to the lower frequency of negative messages in real-world logs. They proposed using the Tomek link technique to balance the two sentiment types. SentiLog is an other approach proposed by Zhang et al. to analyze parallel file system logs and detect anomalies. SentiLog uses a collection of parallel file systems' logging-related source code to train a generic sentimental natural language model. In this manner, SentiLog learns the implicit semantic information based on LSTM developers have embedded within the parallel file system. Like pylogsentiment, SentiLog shows that SA is a promising approach

for modeling unstructured and difficult-to-decode system logs. A2Log⁸² is a two-step, unsupervised anomaly identification method for log data that includes anomaly scoring and decision-making. Initially, a self-attention neural network is employed to assign a score to each log message. Furthermore, it sets the decision boundary by augmenting the existing normal training data without using any anomalous cases. LogELECTRA⁷⁶ is a self-supervised anomaly detection model designed for unstructured system logs. It examines individual log messages without the need for a log parser. The system acquires an understanding of the context of normal log messages by training a discriminator model to identify replaced tokens in changed sequences. During the inference process, LogELECTRA computes an anomaly score for every log message, identifying anomalies as individual anomalies in real-time without needing time-series analysis.

LLM-based unimodal approaches: After demonstrating LLMs' unique and emerging abilities in performing various tasks¹¹⁷, recent studies have proposed using LLMs to detect anomalies from log data. For example, LogFiT³⁵ is an innovative approach for detecting anomalies in system logs. However, it does not rely on predetermined templates or labeled data. Instead, it utilizes a language model based on bidirectional encoder representations from transformers (BERT) to analyze the logs. The system acquires knowledge of language patterns by making predictions on masked sentences using normal log data. LogFiT evaluates the accuracy of predicting the top-k tokens during the inference process to determine a threshold for detecting abnormalities. LogFiT is specifically engineered for easy integration into pre-existing NLP frameworks. LogPrompt⁹³ leverages LLMs to automate log analysis in online scenarios. The system tackles the challenges associated with the restricted ability to understand log anomaly detection results and the ability to handle unseen logs. LogPrompt employs three advanced prompt strategies: self-prompt, chain-of-thought prompt, and in-context prompt. LogPrompt allows for the efficient analysis of log files and identifying abnormal patterns without requiring a large amount of training data. Human assessments demonstrate that LogPrompt's interpretations are valuable and easy to understand, assisting professionals in understanding log data. Pan et al. proposed RAGLog to detect anomalies in log data. RAGLog leverages the retrieval augmented generative (RAG) model with a vector database and an LLM. It avoids the necessity of interpreting logs by directly consuming unprocessed log data. The LLM conducts semantic analysis to compare normal log entries with the query entry, enabling RAGLog to function as a zero-shot classifier that does not necessitate abnormal log samples for training.

Multimodal approaches

This section specifically examines models that utilize multimodal analysis to find anomalies in log data. These approaches are relatively infrequent in comparison to unimodal methods in log analysis. Deep learning employs various techniques, such as MLPs^{42,44,45}, CNNs^{48,51}, RNNs^{36,55,59,61,62,65–67}, autoencoders^{42,69}, GANs⁴², transformers^{83–85}, attention mechanisms^{48,86,88}, and EGNs⁹¹, which are commonly used in these approaches. The three main categories of multimodal techniques mentioned above are early^{51,59,61,62,83,85,86,88,91}, intermediate^{42,45,48,55,65–67}, and late⁴⁴ fusion-based methods. The exceptions to this rule are^{36,69,84}. DeepLog³⁶ is an early example of using deep learning for log anomaly detection. It works by analyzing the sequential relationships between log entries, treating them as a sequence of natural language. Next, learn about the normal log patterns, consisting of a log key and a parameter value vector representing each log entry. DeepLog utilizes two LSTM models to learn about log key sequences and parameter value vectors. In addition, LSTMs are used to forecast future log events. These predictions are then compared to the ground truth to identify any anomalies. Qian et al. proposed VeLog, a method for detecting anomalies in distributed systems using variational autoencoders (VAEs). The process consists of two distinct stages: offline training and online detection. During the offline phase, VeLog gathers log data, analyzes it, and produces the log order matrices and the log count vector matrices representing log executions. The VAE model acquires knowledge about normal sequence patterns based on these matrices. VeLog analyzes and extracts features from new logs during the online phase, generating comparable matrices. Anomalies are identified by comparing the resulting matrices with previously learned patterns. This approach is highly effective when applied to large-scale distributed systems. He et al. proposed UMFLog, a novel unsupervised model for detecting anomalies in log data. UMFLog employs two sub-models to capture the logs' semantic and statistical features, effectively. The first model uses BERT to extract semantic features from log content. In contrast, the second model employs a VAE to learn statistical features, specifically focusing on word frequency in logs. This dual-feature approach improves the model's capacity to identify anomalies without needing labeled data, making it well-suited for practical applications.

Early fusion-based multimodal approaches: Typically, these approaches include extracting meaningful feature vectors from various aspects of log data and constructing a supervector. Finally, they utilize this supervector as the source of knowledge to train their model. For example, Yu et al. add an attention layer following the BiLSTM to assess the value of prior events in predicting subsequent events. They merge log data's sequential and quantitative features with semantic features to achieve effective results. After the sequence, quantitative, and semantic feature vectors have been extracted, they are combined and fed into the model to understand the relationships between the log events. This framework can extract the underlying dependencies from event logs. LogST⁵¹ is a log anomaly detection method that combines semantic and topic features extracted from logs. The process involves utilizing BERT and singular value decomposition (SVD) to extract semantic features. Furthermore, latent dirichlet allocation (LDA) extracts topic features. The combined features are subsequently fed into the improved TCN model, which utilizes weighted residual connections for anomaly detection. Han et al. proposed Log-MatchNet, a method for detecting anomalies in log data designed to handle unstructured and imbalanced log data. It employs log parsing to transform log content information into vectors, utilizing the BERT model for universal feature extraction. The approach involves a matching network to learn similarity scores between input and prototype vectors. The prototype vectors represent a small number of labeled abnormal logs. These vectors are utilized to make generalizations about unknown log samples in a few-shot scenario. Yu et al. proposed LogMS, a log anomaly detection technique that leverages many sources of information and

employs probability label estimation. The system utilizes an multi-source information fusion-based-LSTM network to process semantic, sequential, and quantitative information and a probability label estimation-based gate recurrent unit network to handle pseudolabeled data. The second stage turns on when the abnormalities are not identified in the first stage to optimize efficiency. LogMS efficiently identifies anomalies, even when the log data changes over time. Zhang et al. proposed MultiLog, a multivariate log-based anomaly detection approach for distributed databases. MultiLog initially processes sequential, quantitative, and semantic information from the logs of each database node. This is done using an LSTM model paired with a self-attention in the standalone estimation module. The system utilizes an autoencoder with a meta-classifier to normalize node probabilities and detect abnormalities throughout the entire cluster.

Intermediate fusion-based multimodal approaches: These approaches extract features and process each modality independently until a certain point, at which point the feature maps are concatenated before categorization or decision-making. For example, Li et al. found these challenges unmet by prior methods: (1) actively developing and maintaining software systems change log formats, (2) the trivial monitoring tools may miss the underlying reasons for performance issues. Thus, SwissLog, a robust deep learning-based model for log analysis, is proposed. SwissLog utilizes BERT to embed the semantic information onto the embedding vector and puts the temporal information into another embedding vector. SwissLog feeds to an attention-based BiLSTM model, an intermediate fusion of semantic and time embedding vectors to learn the distinctions between normal and abnormal log sequences. Zhang et al. proposed LogAttn, a method using an autoencoder for unsupervised log anomaly detection. LogAttn begins by parsing log data into structured representations. It generates event count and semantic vector sequences. TCN and deep neural network encoders learn temporal and statistical correlations in log data separately. The decoder reconstructs the log sequence using a latent representation formed by an attention mechanism that weighs feature importance. Comparing the reconstruction error against a predetermined threshold using a model trained entirely on normal log data provides anomaly detection. Niu et al. proposed FSMFLog, an approach for detecting log anomalies by employing multi-feature fusion. FSMFLog overcomes the constraints of previous log parsing techniques by utilizing a prefix tree structure to extract semantic data in word lists rather than relying on typical log templates that frequently fail to capture critical semantics. It commences with preprocessing by removing variables from logs and subsequently grouping log sentences utilizing heuristic strategies. FSMFLog utilizes a bidirectional GRU model that is improved using an attention mechanism. This model utilizes semantic, time, and type features. Yang et al. proposed a log anomaly detection method integrating a self-attention mechanism with a bidirectional GRU model. It utilizes two bidirectional GRU models: one is responsible for processing the log template sequence, while the other one is dedicated to handling the log template frequency vector. The results from both models are concatenated and fed into a self-attention layer, which fuses the features to predict the probability distribution of the subsequent log template. This method detects anomalies when the expected template is missing from the candidate set. Niu et al. proposed a robust log-based anomaly detection framework named WDLLog, which integrates wide and deep learning to address challenges in detecting anomalies in evolving log data. The method begins with an optimized log template extraction process that enhances the traditional Drain algorithm by incorporating semantic embedding and clustering, effectively reducing sensitivity to change in log wording. Following this, WDLLog extracts three types of features from the generated log templates: temporal features, invariant features, and statistical features. Anomaly detection is then performed using a combination of a GRU model with attention mechanisms for temporal features and a gradient boosting decision tree model for invariant and statistical features.

Late fusion-based multimodal approaches: Under these methods, the fusion occurs after each modality has been processed and classified, independently. The results from each modality are then combined to make the final decision. For example, Zhao et al. proposed an approach for detecting web scanning behaviors in online web logs, which are crucial for identifying potential cyber-attacks. It utilizes three lightweight classifiers that analyze distinct features extracted from web traffic logs such as HTTP textual content, status codes, and request frequency. This approach categorizes logs on the source and destination IPs and evaluates the characteristics of these logs to differentiate between scanning and normal behaviors. Each classifier processes specific features. Therefore, textual features are analyzed using an MLP, while HTTP status codes and request frequencies are assessed through support vector machines (SVMs). The classifiers generate probabilities indicating scanning behavior, which are combined using a decision strategy to determine the final classification outcome.

Deep learning for multimodal sentiment analysis

Traditional sentiment analysis procedures often focus on extracting user sentiment from a single modality. MSA incorporates visual and audio modalities into text-based SA. Audio and visual characteristics are employed as they provide a superior ability to better explain or depict sentiment compared to a lengthy list of words. Many studies^{94–103} have been undertaken in this field, most depending on deep learning concepts. The increasing prevalence of deep learning can be attributed to its ability to boost complex processing, thereby improving the automation process. From a multimodal learning standpoint, comprehensive surveys^{118,119} have outlined foundational principles, challenges, and open questions in multimodal machine learning. These works emphasize systematic integration, balancing, and interaction of modalities - all central issues addressed in CoLog. Moreover, Qiu et al. proposed a chained interactive attention mechanism for multimodal sentiment analysis, further validating that multimodal fusion strategies developed in the sentiment domain can inspire advances in anomaly detection. Delbrouck et al. proposed a transformer encoder architecture that fuses any information about modalities. The model at hand utilizes joint-encoding to concurrently encode each modality, with modular co-attention controlling how a modality attends to itself. Hu et al. proposed a framework called UniMSE. The UniMSE framework is a novel method for unified multimodal sentiment analysis and emotion recognition in conversations. It reformulates tasks into a generative format, integrates acoustic, visual, and textual features, and employs inter-modal contrastive learning to minimize intra-class variance and maximize

inter-class variance. Li et al. proposed a multimodal sentiment analysis model using transformer architecture and soft mapping. The transformer layer utilizes the attention layer to map modalities, while the soft mapping layer uses stacking modules for multimodal information fusion. This model addresses data interaction issues in multimodal sentiment analysis by considering the relationships between multiple modalities. The TETFN⁹⁷ is a method for multimodal sentiment analysis that enhances video sentiment recognition by integrating textual, visual, and audio modalities. It uses preprocessing, feature extraction, LSTM networks, and TCNs to encode contextual information and generate unimodal labels. The core of TETFN is a text-enhanced transformer module that leverages a text-oriented multi-head attention (MHA) mechanism to incorporate textual information into the visual and audio modalities, facilitating effective pairwise cross-modal mappings. Ahuja et al. proposed a novel multimodal sentiment analysis method for images containing textual information. Initially, it employed a recognition system to extract text from images using the Google Cloud Vision API, which is noted for its high accuracy in optical character recognition (OCR). The extracted text is preprocessed and analyzed with a RoBERTa-based model for textual sentiment analysis. Concurrently, visual sentiment analysis uses a transfer learning approach to capture visual features. The method integrates the outputs from both analyses through a weighted fusion strategy, enhancing the overall sentiment classification accuracy. Liu et al. proposed a robust multimodal sentiment analysis model that uses a modality translation-based approach to handle uncertain missing modalities. It involves translating visual and audio data to text modality, encoding text, and fused into missing joint features (MJFs). The transformer encoder module then encodes these MJFs to learning long-term dependencies between modalities. Therefore, the sentiment classification is based on the transformer encoder module's outputs.

Beyond log-specific approaches

Beyond log-specific approaches, multimodal anomaly detection has been explored in other domains such as video understanding. For instance, Su et al. a proposed semantic-driven dual consistency learning for video anomaly detection that leverages semantic-driven representations to align appearance and motion features, enhancing anomaly localization under weak supervision. Similarly, Su et al. proposed federated weakly-supervised video anomaly detection with a mixture of local-to-global experts that employs mixtures of experts within a federated learning framework to address distributed and heterogeneous data scenarios.

While these methods target video modalities, they share conceptual parallels with CoLog in terms of semantic-guided fusion and cross-modal consistency learning. Inspired by such principles, CoLog's collaborative transformer integrates semantic and sequential log modalities through impressed attention and the Modality Adaptation Layer (MAL), enabling unified detection of both point and collective anomalies.

Our method: a cut above the rest

According to the literature cited in Sect. "Deep learning for anomaly detection from logs", diverse deep learning techniques are progressively being employed to detect anomalies in log records, which we classified into two general categories and six subcategories. Figure 2 demonstrates a concise overview of log-based anomaly detection. CoLog falls under the umbrella of multimodal-based approaches.

However, our method differs as we aim to use the redundancy of modalities within system logs based on MSA and CT. CT can be utilized to supplement diverse modalities. When data are deficient from a particular modality, data from another modality is employed. Therefore, these transformers enable the extraction of various information from modalities and interactively encode modalities.

The strong performance and generalization of sentiment-based log analysis models, as claimed by^{38,52,53,58}, makes well-trained models more robust to the evolution of log statements. We offer the CoLog framework that developed based on the SA of logs. But, unlike^{38,52,53,58}, our model employs a more complex architecture for MSA-based anomaly detection. By evaluating the sentiment of log events in dependence on their background, CoLog can analyze both point and collective anomalies. Moreover, CoLog can distinguish similar events by utilizing auxiliary information from the background of their occurrence, as depicted in Figure 1. In the following sections, we will demonstrate that the context of event logs can also be utilized.

In contrast to reviewed methods, CoLog relies on modality interaction within the logs. It does not concat different modalities into a super vector in any way. The challenge at hand is tackled through impressed attention based on CT. This form of attention is beneficial to discovering the correlation between two modalities. Since

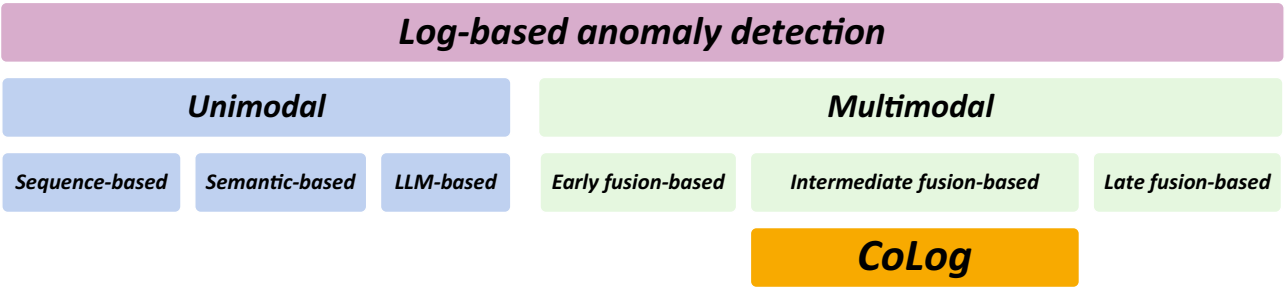


Fig. 2. A comprehensive overview of the existing literature dealing with the approaches for identifying anomalies in log data.

deep learning models are commonly denoted as black-box models due to their complex nature, which surpasses human comprehension, comprehending the reasoning behind each decision is a task that exceeds human cognitive capacity. The utilization of the impressed attention mechanism has the potential to augment the interpretability of our log anomaly detection model. This is because the attention scores, which are computed based on the impressed attention, can offer valuable insights into the model's cross-modal decision-making process. CoLog projects the results of modalities into a latent space above the transformer blocks for intermediate fusion and executes the encoding process, concurrently. Therefore, forming an optimal latent space is crucial due to the inherent incompatibility of various modalities.

Furthermore, the CoLog architecture enables the identification of point and collective log abnormalities using a unified framework. According to our current knowledge, CoLog is the first approach that performs this task.

Preliminaries

This section discusses the task's definition and the rationale for employing the transformer model to accomplish it. It also covers the assumptions and threat model used to evaluate potential threats.

Task definition

This task aims to conduct a log analysis to detect anomalies during the system's execution. The scope of the analysis will include analyzing logs' information to detect abnormalities. A machine learning algorithm will be used to analyze these data. The performance of the anomaly detection method will be evaluated by measuring some metrics, such as accuracy, precision, recall, and F1-score, to ensure that the method does not generate too many false alerts.

When conducting multimodal sentiment analysis, researchers look at the dialogue from various perspectives (modalities) to determine the participants' sentimental states. The three most commonly utilized modalities are textual, audio, and visual. In the same vein, logs possess a nature that, akin to human conversations, conveys information from different aspects, such as statistical, temporal, semantic, and information based on the event's sequence. However, the semantic and sequence modalities in logs are the most crucial ones to consider. Besides, to detect anomalies from logs, we can implement sentimental pipelines to classify sentiments wherein recognizing negative sentiments alerts anomalous situations. In Fig. 3(a), log messages' negative and positive sentimental states are presented in red and green, respectively. Based on these observations, we propose to apply multimodal sentiment analysis to the log anomaly detection task. Figure 3(b) and (c) respectively depict the semantic and sequence modalities. These figures show that semantic vectors of log events are derived based on log messages. These vectors generate sequence vectors based on the window size. In this case, the window size determines the length of the sequence vectors we create. These vectors would serve as input for a transformer-based multimodal framework that learns cross-modal dependencies and predicts point and collective log anomalies.

Transformers: why?

Transformers exhibit a notable advantage in their processing speed when dealing with a sequence. They focus more on the crucial components, resulting in enhanced speed compared to alternative models. Also, transformers exhibit multiple benefits in the context of multimodal sentiment analysis. Transformers can enhance models' robustness in multimodal sentiment analysis, thereby constituting an additional benefit. Another advantage is their ability to incorporate impressed attention into their architecture to extract cross-modal interactions within non-aligned multimodal data. The significance of this matter lies in the complexity and multimodality of most real-world information. Considering transformers' benefits, we decided to base our multimodal model around them.

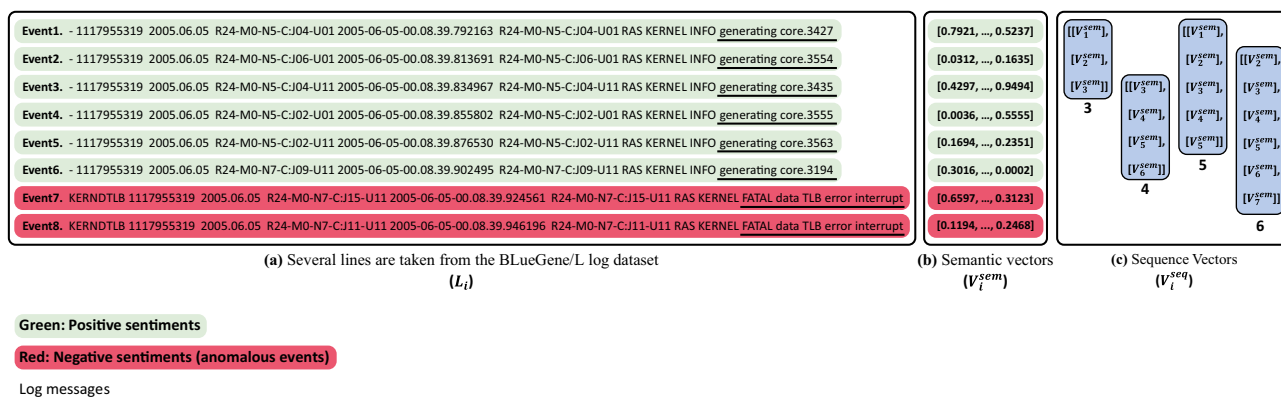


Fig. 3. (a) Several lines extracted from the BLueGene/L log dataset.; (b) Semantic modality: The semantic modality is constructed using the extracted semantic vectors from log events' messages.; (c) Sequence modality: The construction of the sequence modality involves appending semantic vectors into sequence vectors based on window sizes of 3, 4, 5, and 6.

Assumptions

Anomaly detection from the log strongly depends on the quality of the log. Multiple categories of assaults exist that may not be recorded in system logs. Suppose certain auditable occurrences, such as login attempts, are not duly recorded. In that case, it is plausible that CoLog may not locate such forms of abnormalities. Another type of attack that may not be recorded in logs is a log forging or log injection attack. The aforementioned is a man-in-the-middle assault in which an unauthenticated user interposes between the application and the server. Therefore, it is assumed that any attempt by an adversary to modify the system logging behavior is not feasible. In a manner analogous to that of pylogsentiment, we consider log entries like `[c010ce54>] mtrr wrmsr+0xf/0x2e` in a kernel log as having a positive sentiment during training if they do not provide a human-readable log message. It is essential to clarify that our analogy between anomalies and “negative sentiments” does not imply a direct mapping to human emotions. Instead, it serves as an analytical abstraction: anomalies are deviations from expected behavior (negative polarity), while normal and benign events align with expected states (positive polarity). Unlike human sentiment, which emerges from subjective perception, log polarity is determined contextually by semantic embeddings and sequence patterns. For instance, a “restart” event can be benign in maintenance scenarios but anomalous when it occurs unexpectedly.

Threat model

An anomaly is a data instance that diverges from the normal pattern. Various abnormalities exist, including point, collective, and contextual anomalies. A point anomaly refers to a singular data point that exhibits a significant deviation from the overall pattern of the dataset. A group of data instances can form an anomalous pattern known as a collective anomaly. A contextual anomaly is when a given data instance exhibits abnormal behavior only in a particular context while classifying normal in others. A point or collective anomaly can be classified as contextual when examined in a specific context. CoLog identifies point and collective anomalies through a unified framework based on CT. This involves evaluating each log entry for its sentiment based on the background or context in which it occurred and flagging any entry that exhibits negative sentiment as an anomaly.

Method

This section explains architecture and data flow of CoLog, the specific procedures CoLog uses to gather and examine log data to look for anomalies. It includes the conditions under which data were collected, labeled and preprocessed, guidelines that specify how anomaly detection task should be carried out, and DL methodologies and techniques utilized for automatic log-based anomaly detection based on MSA.

Overall architecture

Figure 4 depicts CoLog’s training framework and gives an overview of its architecture. Like most log anomaly detection techniques, CoLog begins by preprocessing raw logs. First, it extracts semantic and sequence modalities from the raw-valued logs. Before feeding the embedded vectors of modalities into the model, the Tomek link approach addresses the class imbalance issue.

Two transformer blocks called collaborative transformers with a modified GA mechanism called impressed attention are applied to these modalities. CoLog comprises a stack of T identical CT blocks, but each one has a different set of training parameters. Each modality encoder comprises a multi-head impressed attention (MHIA), MLP, modality adaptation layer (MAL), and Layer Normalizations (LNs). Modalities fed into the matching modality encoder in CT. The MLP layer, which employs a nonlinear activation function, receives the results of computing the attention scores from the attention layer. Each encoder has a residual connection and LN after the MHIA, MLP, and MAL layers. The results of the encoding blocks are fed into the balancing layer (BL) to determine the sentiment of a log message, where soft attention and fusion are performed. Remember that the negative sentiment is connected to any potential abnormal behaviors.

Due to the information sharing between different modalities in CT, the suggested architecture thus conveniently learns the shared representation between modalities.

Preprocessing

In the preprocessing phase, the log parsing procedure extracts a specific element, denoted as a log message, from a log file. Then, every log message is segmented into tokens and transformed into lowercase. The process involves constructing a lexicon based on the training dataset, resulting in a dictionary comprising V_{size} distinct terms. After that, each word is embedded in a 300-dimensional vector. In cases where a term from the validation or testing dataset is not present in our extracted vocabulary, we substitute it with the “UNK” token. In addition, irregularities exist in the length of individual log messages. Consequently, if the log message’s length falls below the specified threshold, it is padded with zeroes. Conversely, in cases where the length exceeds the established threshold, we truncate to abbreviate it to the predefined message length.

Log parsing

Each log entry contains various separate parts. Log parsing is the act of transforming unstructured log data into a structured format to enable machine interpretation. Every log file is parsed to convert it from an unstructured form to a structured format with entities like timestamp, service name, level information, and log message. We must separate the log message from the log entry because it is the only entity with the sentiment. In addition, each log message is transformed into 384-dimensional vectors using SBERT and stored separately from the log messages. We employ the nerlogparser¹²³ and Drain¹²⁴ log parsers in our implementation. Indeed, each dataset is processed and parsed using its corresponding parser.

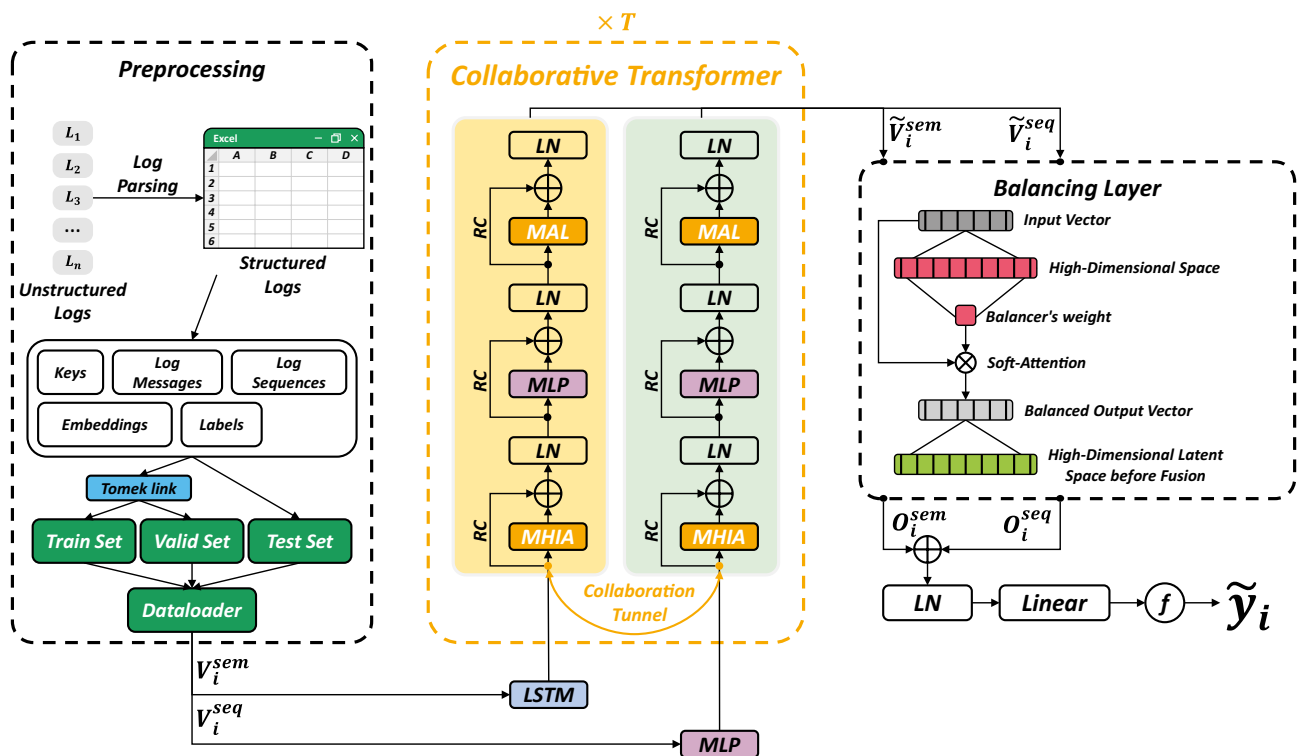


Fig. 4. The overview of CoLog. Light green and gold colors demonstrate modality encoders. Each encoder in the collaborative transformer consists of MHIA, MLP, MAL, and LNs. MHIA and MAL are multi-head impressed attention and modality adaptation layer modules, respectively. The preprocess layer transforms unstructured logs into easily understandable data for the model. The purpose of the balancing layer is to regulate the influences of different modalities when calculating the final results.

Log parser: drain

One of the representative algorithms for log parsing is the Drain. Drain employs a fixed-depth parse tree, which encodes specifically created parsing rules, to expedite the parsing procedure. Drain preprocesses logs first using regular expressions and user-defined domain knowledge. Second, Drain begins with the preprocessed log message at the root node of the parse tree. The parse tree's first layer nodes represent log groups whose log messages are of various lengths. Drain uses a concept of distance called token similarity to determine how similar the log messages are to one another. The tokens in the first positions of the log message are used to choose the subsequent internal node. Drain then determines whether to add the log message to an existing log group by calculating the similarity between each log group's log message and log event. Drain is quite effective and accurate. It can also parse log data in real-time, which makes it well-suited for log anomaly detection. Additionally, Drain is scalable, making it suitable for large-scale environments.

Log parser: nerlogparser

The nerlogparser utilizes named entity recognition (NER), a technique employed to extract named entities from text. Nerlogparser identifies named entities in log files as words or phrases that include common fields seen in a log entry, such as timestamp, hostname, or service name. Named entity extraction is the process of recognizing each element in a log entry. To perform NER, nerlogparser employs BiLSTM. The primary advantage of the nerlogparser is its ability to automatically parse log data using a pre-trained model. Thus, there is no necessity to establish any rules or regular expressions. The nerlogparser can parse a wide range of log files due to its training on multiple log types.

Task formulation

We employ the notation L_i^m , $m \in \{sem, seq\}$ to represent the unimodal raw-valued log modalities extracted from the log messages $\{L_{i-w}, L_{i-(w-1)}, \dots, L_{i-1}, L_i\}$, where w is the window size and the notation $\{sem, seq\}$ specifies the two different types of modalities: semantic and sequence. The parsed raw log messages, which include sentimental words, are defined in mathematical terms as $L = \{L_1, L_2, \dots, L_{|L|}\}$, where the raw log message L_i extracted from the log entry i , $|L|$ denotes the total number of log messages. The architecture of model blocks, model parameters, and label space are unified through task formulation, wherein two transformer blocks receive semantic and sequence modalities. These transformers learn representations of log data. Finally, our model attempts to predict the integer value $\tilde{y}_i \in \{0, 1\}$ needed to classify log entry i .

Task formulation involves two subsections. formulating input features describes the process of transforming raw log modalities into input feature vectors for the model. However, label formulation applies MSA for log anomaly detection tasks by transforming log message labels into a space representing sentimental states.

Formulating input features

The comprehension of events' sentiments mainly relies on the semantic information in log messages. Based on this observation, constructing semantic modality involves preprocessing and segmenting each raw log message into tokens from t_1 to t_n as raw words. Subsequently, each token is added to a list that retains preprocessed log messages represented as segmented text:

$$L_i^{sem} = [t_1, t_2, \dots, t_n]. \quad (1)$$

where,

$$n = \text{number of tokens} \quad (2)$$

After this, word embedding vectors are acquired by utilizing word2vec¹²⁵ to create embedding vectors of tokens in L_i^{sem} .

Furthermore, the raw sequence features of every log message are transformed into numerical sequential vectors. Log sequence vectors can be produced in two procedures: background and context. As illustrated in Fig. 5, generating background and subsequent event lists for every log entry is possible. A background sequence vector can be generated by concatenating background event vectors. The resulting vector is a $(W \times k)$ -dimensional, where W represents the window size. Based on the above mentioned concepts, L_i^{seq} can be defined as follows:

$$L_i^{seq} = [[L_j]]. \quad (3)$$

where,

$$j \in \{i - W, i - (W - 1), \dots, i - 1\} \quad (4)$$

Similarly, the context sequence vector, constructed by concatenating the background and subsequent event vectors, has a dimension size of $2W \times k$.

$$L_i^{seq} = [[L_j]]. \quad (5)$$

where,

$$j \in \{i - W, i - (W - 1), \dots, i - 1, i + 1, \dots, i + (W - 1), i + W\} \quad (6)$$

The decision-making process regarding whether to use Equation 3 or Equation 5 is based on the type of sequence modality that goes through background or context extraction operations.

Label formulation

The objective of CoLog is to utilize MSA to detect log anomalies and predict the sentiment reflected by log message i . In SA, the numerical value 1 is commonly used to denote positive sentiments, while 0 is typically employed to indicate negative sentiments. The task of identifying negative sentiments can be considered

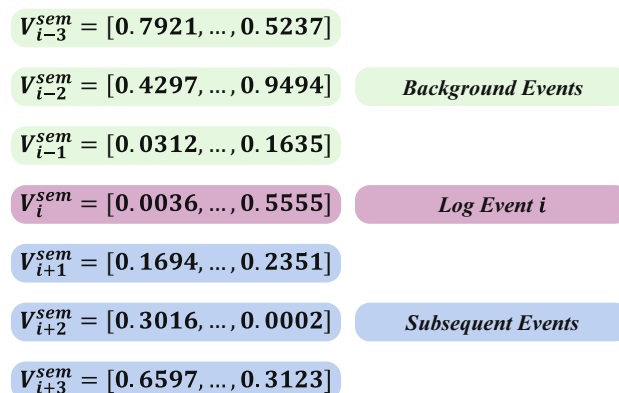


Fig. 5. Illustration of differences between background and subsequent event vectors. Background and context sequence vectors are constructed based on background and subsequent event vectors. In mathematical terms, $B_i = [V_{i-3}^{sem}, V_{i-2}^{sem}, V_{i-1}^{sem}]$ and $C_i = [V_{i-3}^{sem}, V_{i-2}^{sem}, V_{i-1}^{sem}, V_{i+1}^{sem}, V_{i+2}^{sem}, V_{i+3}^{sem}]$, where V_i^{sem} is the semantic vector of the log message i extracted by SBERT, B_i is the background sequence vector of log message i , and C_i is the context sequence vector of log message i .

equivalent to the detection of log anomalies. Consequently, every instance within the dataset is assigned a label of either 1 or 0, corresponding to its classification as a normal or abnormal sample. In this manner the universal label $y_i = \{y_i^p\}$ comprises the polarity of log message i , where $y_i^p \in \{0, 1\}$.

Unlabeled datasets are labeled using words that reflect negative sentiment in the same dataset. The list of negative sentimental words for the unlabeled datasets is based on the conditions proposed in pylogsentiment³⁸.

Leveling the playing field: class imbalance

The issue of class imbalance is a prevalent concern in machine learning, characterized by an unequal distribution of classes within the training data. The potential outcome of this scenario is the development of models that exhibit bias toward the majority class, resulting in suboptimal performance on the minority class. Various techniques have been devised to address the issue of class imbalance, including data-level approaches, e.g., over-sampling methods^{126,127} and under-sampling techniques^{128,129}, and algorithm-level approaches, e.g., ensemble methods^{130,131}. The techniques mentioned above are designed to enhance the efficiency of machine learning models when dealing with imbalanced datasets. This is achieved by either fixing the data imbalance or adapting the learning algorithm to consider the distribution of classes.

For class balance, we employ the under-sampling technique called the Tomek link¹²⁹. The Tomek link is utilized for under-sampling to generate a novel distribution of the majority class. The log data frequently contains a repetitive majority class, which can be regarded as noise. Consequently, we opted for this approach, which is commonly employed to eliminate noisy and borderline majority class instances. A Tomek link is established between two data items belonging to different classes that are the nearest neighbors to each other. Assuming the entire dataset is denoted as $L = \{L_1, L_2, \dots, L_{|L|}\}$, the balanced dataset $L_{balanced}$ can be defined mathematically as follows:

$$L_{balanced} = tomeklink(L). \quad (7)$$

where $tomeklink(\cdot)$ is a function aimed at removing Tomek links (L_i, L_j) from dataset L . (L_i, L_j) is a Tomek link provided that there is no L_k whose Euclidean distance from any member of (L_i, L_j) is not less than the Euclidean distance of L_i and L_j . Eliminating Tomek links continues until every pair of nearest neighbor vectors belongs to the same class. It is crucial to note that L_i and L_j belong to distinct classes.

Collaborative transformer

The attention layer is CoLog's most crucial component. To determine the mapping relationship among modalities of log data, CT is created by modifying the attention layers of two identical transformer blocks that concurrently learn data representations in an end-to-end manner. As a result, when learning information from one modality, we use the information from the other modality as guidelines enabled by the MHIA mechanism.

The attention mechanism of an unimodal transformer encoder is defined as follows:

$$attention(Q, K, C) = softmax\left(\frac{QK^T}{\sqrt{d}}\right)C. \quad (8)$$

where Q, K , and C are the respective queries, keys, and contexts. We implement the self-attention mechanism with $Q = K = C$. Query, key, and context vectors enable the model to understand the relative importance of all words within the context of the full sequence. The operation QK^T produces a squared attention matrix that contains the correlation between row of input matrix V^m , $m \in \{sem, seq\}$ if this input has a size of $N \times k$. The expression $\sqrt{d}$ is a factor used to adjust the scale where $d = k$.

The concept of stacking several self-attentions attending data from various representation sub-spaces in different positions is known as MHA as follows:

$$MHA(Q, K, C) = concat(h_1, h_2, \dots, h_h)W_o. \quad (9)$$

where,

$$h_i = attention(QW_i^Q, KW_i^K, CW_i^C) \\ i \in \{1, 2, \dots, h-1, h\} \quad (10)$$

where $W^Q, W^K, W^C \in R^{d \times d}$. We stack h layers of self-attention to perform MHA to learn complex patterns between events. The projection into the balancing layer for classification follows encoding through the transformer blocks, with the output of each transformer block being used.

As illustrated in Fig. 4, integrating MHIA in CT involves substituting MHA with MHIA in each modality encoder, which is demonstrated with light green and gold colors. The architecture of MHIA is illustrated in Fig. 6. MHIA requires concurrent encoding across utilized modalities. In addition, the MHIA implementation includes computing the attention scores of the at-hand modality by utilizing the Q vector from the at-hand modality and the K and C vectors from the secondary modality using the MHA mechanism. Each modality encoder transformer in the CT architecture contains a corresponding MHIA block.

Concurrent-encoding

The concurrent-encoding process means simultaneous encoding of each modality. As mentioned above, this concept implies that the encoding block used for a particular modality is not unrolled before moving to another

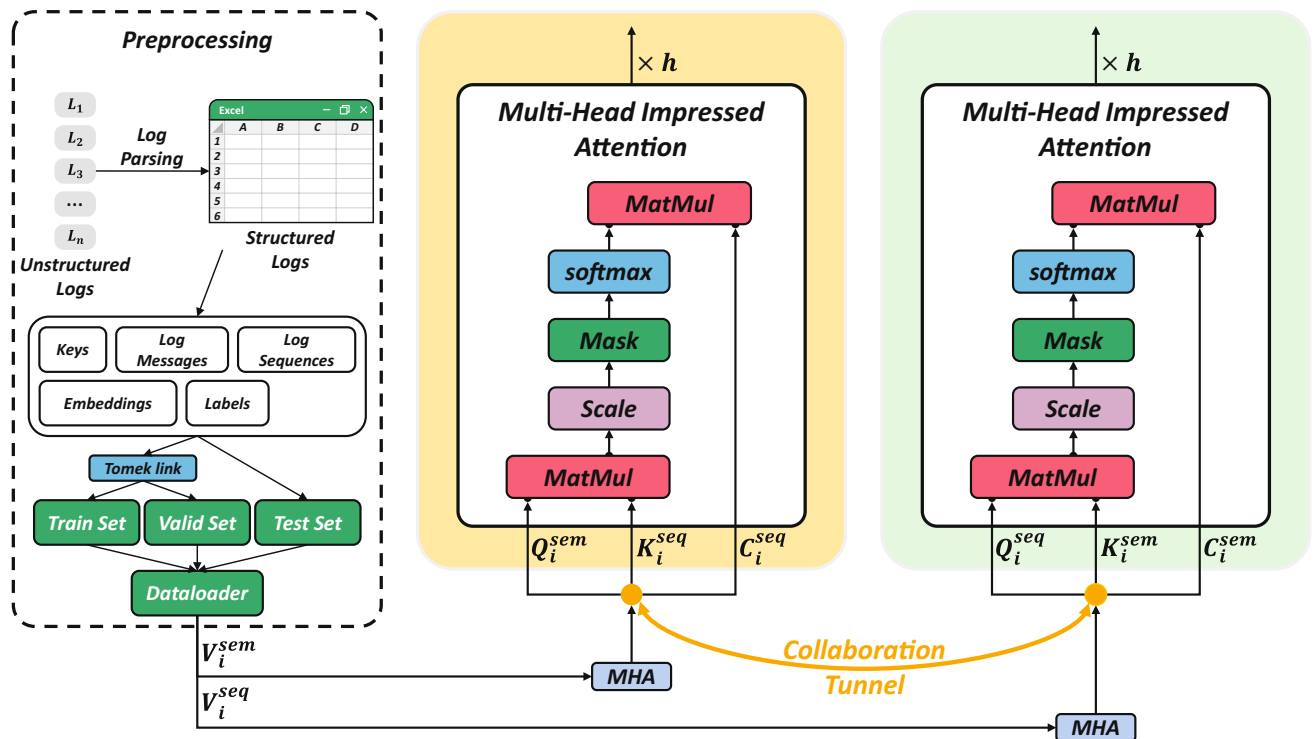


Fig. 6. The architecture of the multi-head impressed attention layer. The MHIA process involves calculating the attention scores of the current modality through the MHA mechanism, using the Q vector from the current modality and the K and C vectors from the secondary modality. According to MHIA architecture, various modalities are encoded concurrently.

modality. The projection into latent space for classification follows encoding through the transformer blocks by the balancing layer.

Modality adaptation layer

Since different modalities are collaboratively encoded in the CT architecture, it is essential to ensure that the extracted representations are free of unnecessary information. Because logs have several modalities, encoding each modality using knowledge from other modalities is crucial in obtaining an improved and more informative representation of that modality. However, due to the intrinsic variations between different modalities, encoding them collaboratively can result in inconsistencies and impurities in the representation of each modality. MAL is proposed as a solution to address these challenges. MAL comprises N soft attention layers, which their corresponding outputs are stacked. Each modality V^m , $m \in \{sem, seq\}$ is initially transformed into a new high-dimensional representation space V_{high}^m to start MAL. This transformation is calculated using Equation 11 as follows:

$$V_{high}^m = V^m W_{high}^m. \quad (11)$$

where $V^m \in R^{N \times K}$ is the input matrix on which CoLog performs MAL and $W_{high}^m \in R^{k \times 2k}$ is a transformation matrix that embeds V^m into a higher dimension shared among all MALs.

In the second phase, weights for each node in the sequence must be generated from the acquired high-dimensional space to implement soft attention and extract a pure and global representation of every node in the sequence. To do this, we need the w_i^{node} , $i \in [1, N]$'s. each w_i^{node} must have a size of $1 \times 2k$, while the set of vectors $\{w_i^{node}\}$ is unique to each node in input sequence. The weights of each node are derived using Equation 12, based on the following notions.

$$w^m = softmax(V_{high}^m (W_{low}^m)^T). \quad (12)$$

where,

$$W_{low}^m = concat(w_i^{node}). \quad (13)$$

where W_{low}^m is a transformation matrix that extracts weights from high-dimension space for each node.

Equation 14 computes the soft attention of the input matrix based on the weights acquired from the previous phase as follows:

$$\tilde{V}_i^m = \text{softattention}(V^m) = \sum_{j=0}^N w^m[i][j] \odot V^m[j]. \quad (14)$$

where $\tilde{V}_i^m$ represents the computation output obtained from an individual node within the sequence and $\odot$ demonstrates the Hadamard product.

The output of the MAL for an input matrix V^m with a sequence length of N can be defined as stacking all $\tilde{V}_i^m$ vectors according to following relation:

$$\tilde{V}_{out}^m = \text{layernorm}(\tilde{V}^m + V^m). \quad (15)$$

where,

$$\tilde{V}^m = \text{stacking}(\tilde{V}_1^m, \tilde{V}_2^m, \dots, \tilde{V}_N^m). \quad (16)$$

The transformation matrix W_{high}^m , shared among all nodes, maps each node in the input sequence to a high-dimensional space, as illustrated in Fig. 7. The softmax function and vector w_i^{node} allocate the weight $w^m[i]$ for each node i , which is distinct for every node. The final output is obtained by computing the weighted sum of all nodes, which is then fed into the balancing and classification layers to make the final decision. We perform this to enable each modality to recognize the corresponding cross-node representations and remove impurities of each modality. By completing this task, every node vector can elicit important information from other nodes' respective representations, contributing to each node's global sentiment polarity.

Balancing layer

At the end of the modality transformer blocks, we add a balancing layer that projects the modality into a new representation space. Due to the inherent differences between various log modalities, CoLog summarizes feature vectors of each modality in the same semantic space. Therefore, as seen in Fig. 4, the learned outcomes from each modality encoder are projected into a latent space for meaningful fusion where CoLog interprets the feature vectors of each modality in a space with high dimensions. Furthermore, since different modalities may contribute different importance to determining the normality or abnormality of the input, we extract the balancer's weight for each modality from their respective high-dimensional representations. Based on the balancer's weights, we can perform soft attention to balance the contribution of modalities before fusion. The balancing layer is the same as MAL of size 1, with the distinction that the resulting vectors are projected into a latent space to ensure a meaningful fusion. Integrating a latent space in BL is due to the different nature of the modalities.

The output of the balancing layer for semantic and sequence modalities is demonstrated using the symbols O_i^{sem} and O_i^{seq} , respectively.

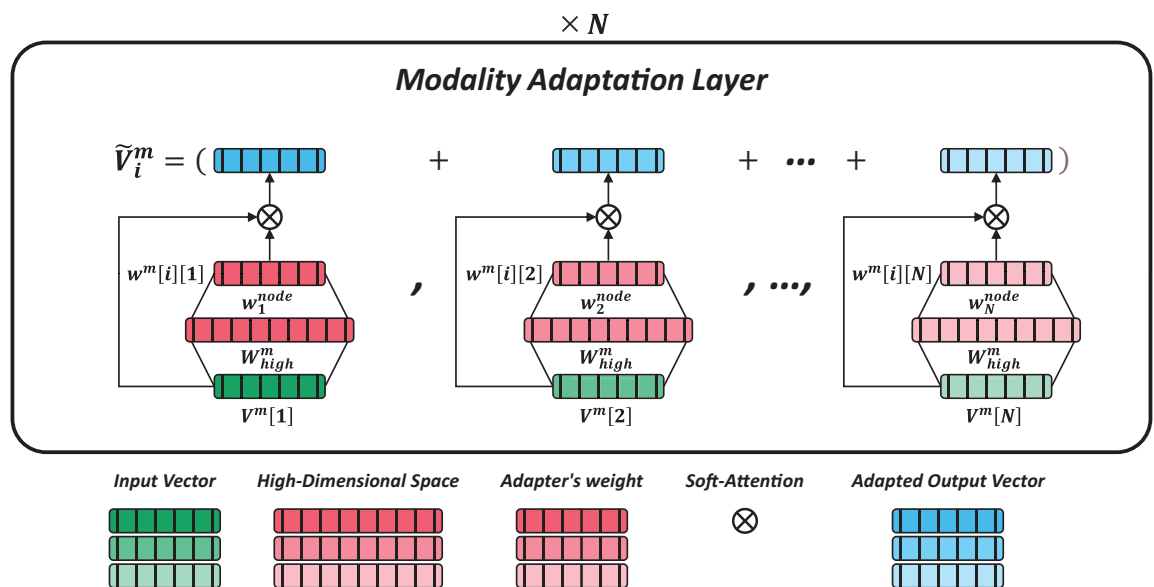


Fig. 7. The architecture of modality adaptation layer. MAL achieves an overall representation for each modality by assigning weights to each node in the input sequence. It can also remove impurities of the modalities that are encoded in a collaborative manner.

Classification

Finally, after being projected to latent space, balanced feature vectors of each modality are summed, normalized, and transformed into 2-dimensional vectors to apply the activation function. According to Fig. 4, the vectors corresponding to each modality undergo element-wise summation following the successful computation of all transformer blocks. Subsequently, these vectors are used to predict each input sample by MLP and layer normalization, as demonstrated by:

$$y_i = W_{classification}(layernorm(O_i^{sem} + O_i^{seq})). \tag{17}$$

where $W_{classification}$ denotes the learning matrix of MLP.

Anomaly detection

If various CoLog inputs are provided to the model within a single variable, the CoLog input can be referred to as $I = \{keys, V^{sem}, V^{seq}, embeddings, labels\}$. CoLog receives I and tries to predict the set of labels y . The anomaly detection phase is similar to the training process. It is important to note that CoLog can detect anomalies in two distinct manners. This involves prioritizing the detection of point anomalies only or the detection of point and collective anomalies.

Point anomaly detection

Based on the methodology provided in Section 4, CoLog takes input I and attempts to make predictions about them during the point anomaly detection procedure. This procedure aims to receive an individual log entry by CoLog, analyze it in collaboration with its background or context, and determine its polarity.

Point and collective anomaly detection

Point and collective anomaly detection through a unified framework entail receiving input by CoLog and attempting to ascertain the polarity of an individual log event and its background or context polarity. From this concept, it can be inferred that in this scenario, the task of CoLog is a 4-class classification. These classes encompass the abnormality of an event, the abnormality of the event and its background or context, the abnormality of just the background or context of the event, and the normality of both the event and its background or context. Hence, it is imperative to label the background or context of the events in this particular scenario.

A log event sequence $L = \{L_1, L_2, \dots, L_{|L|}\}$ is abnormal if one of its log messages reflects a negative sentiment. Based on this, the background or context of each log event can also be labeled. Finally, based on the methodology provided in Sect. “Method”, CoLog takes input I and generates predictions during the point and collective anomaly detection procedure. This procedure aims to receive an individual log entry by CoLog, analyze it in collaboration with its background or context, and determine its polarity as well as its background or context polarity.

Experiments

This section will begin by describing the OS log datasets. Subsequently, we will discuss the experimental setup of CoLog and the evaluation results of the proposed model.

Log datasets

Experiments are conducted on benchmark OS log datasets, including the publicly available BlueGene/L, Hadoop, and Zookeeper datasets. Table 2 presents a comprehensive statistical information of all datasets utilized for evaluating CoLog’s performance.

Spark¹³², Honey5¹³³, and Windows¹³² datasets are not used during CoLog training. These datasets are used to test the generalizability and robustness of CoLog. Spark is a management solution for handling large amounts of data. Normal and abnormal Spark system actions are both represented in the logs obtained from 32 hosts. Honey5 originates from the Forensic Challenge 5 in 2010, which the Honeynet Project conducted. The dataset

Dataset	# Lines	# Positive records	# Negative records	Data size
Spark ¹	33, 236, 604	31, 513, 147	1, 723, 457	2.75 GB
Honey5 ²	124, 386	67, 798	56, 588	12.6 MB
Windows ²	25, 000	18, 599	6, 401	3.48 MB
Casper	11, 086	9, 874	1, 212	930 KB
Jhuisi	11, 737	9, 063	2, 674	0.98 MB
Nssal	107, 093	91, 349	15, 732	8.53 MB
Honey7	8, 712	8, 162	550	734 KB
Zookeeper	74, 380	25, 873	48, 507	9.95 MB
Hadoop	394, 308	382, 870	11, 438	48.61 MB
BlueGene/L	4, 747, 963	4, 399, 486	348, 477	708.76 MB

Table 2. A summary of publicly available OS log datasets. ¹The provided datasets are intended to evaluate the robustness of CoLog. ²The provided datasets are intended to evaluate the generalization of CoLog.

provided is an instance of a Linux OS that has been compromised. As the last dataset, we use the Windows dataset, obtained by consolidating many logs from the CUHK laboratory computer operating on Windows 7. The Windows OS implemented a component-based servicing (CBS) mechanism to facilitate programs' secure and controlled installation processes.

The first dataset of the other seven datasets of this collection used in the model training process was taken from a disk image provided by Digital Corpora and given the name nps-2009-casper-rw¹³⁴. A bootable USB was used to create a dump of the ext3 file system. The OS logs from a machine running the Linux OS are included in this dataset. The digital forensic research workshop (DFRWS), an annual security conference, provides the second and third system log datasets we use to train CoLog. They presented an OS log forensics challenge in 2009. The DFRWS forensic challenge 2009 studied log files to find an intruder who illegally communicated classified material. This case involves jhuisi¹³⁵ and nssal¹³⁵ hosts. Two Linux-based Sony PlayStation 3 hosts. Additional OS logs were obtained via the honeynet forensic challenge 7 (Honey7)¹³⁶. Honey7 includes a disk image of a compromised Linux server. The cloned disk images contained the directory `/var/log/` for all datasets. Then, authentication, kernel, and system logs are obtained. Zookeeper manages distributed systems. The CUHK laboratory collected Zookeeper logs¹³² from 32 hosts for 26 days. Big data tool Hadoop distributes jobs across machines. A Hadoop cluster with 46 cores on five machines created the Hadoop log dataset¹³⁷. In this dataset, machine downtime and network disconnections are anomalies. The 131,072 processor, 32,768 GB memory BlueGene/L supercomputer at Lawrence Livermore National Laboratories (LLNL) provides an open dataset¹³⁸. The records comprise alert and non-alert, where alerts signify abnormal behaviors.

Experimental setup

This section describes the hardware and software configurations of the system we used to run experiments. We then discuss CoLog's hyperparameters and the metrics utilized to evaluate CoLog.

System configs

We use a Colab pro machine with 12 cores on the central processing unit (CPU), 83.5 gigabytes of random-access memory (RAM), and an NVIDIA A100 40GB graphics processing unit (GPU) to execute the experiments for both the CoLog and the other methods.

Python 3.10.12 is used to develop CoLog, and PyTorch¹³⁹ 2.0.1+cu118 is used as the backend. We use the imbalanced-learn¹⁴⁰ library to implement the Tomek link and evaluate it against other class-balancing techniques. The Ray library¹⁴¹ in Python was utilized to determine the model's optimal parameters. The sentence-transformers library¹¹² was also employed to extract sentence embedding vectors from the log messages.

Hyperparameters

The Ray Python library chooses the most optimal hyperparameters for CoLog. Using the above mentioned library, 108 distinct configurations of CoLog were analyzed. In addition, an examination of window size, class imbalance, and the training ratio has been conducted independently to determine the optimal values for each hyperparameter. The number of CT layers is 2, and the number of heads is 4. The batch size is 32. The learning rate for training the CoLog is set to $5e-5$, whereas the learning rate decay is set to 0.5, and the learning rate will decrease three times at this rate. The Adam optimizer method is employed. Adam is considered suitable for this case due to its minimal RAM requirements. The training process has a maximum epoch of 20 and an early stopping criterion set to 5. The training process will be terminated if no further improvement is shown after 5 epochs. The model warm-up process consists of 5 epochs. The dropout rate of 0.1 is used. The lengths of the semantic and sequence modalities are 60, whereas the embedding size of the word vectors is 300. The hidden size of 256 is used for the CoLog model. The projection vector size is also 2048. The sequence modality type is set to context, and a window size 1 is used. Dimensions of sequence modality are set to 384. We employ a holdout validation protocol. The datasets were divided into three pieces based on the following proportions: 60% for training, 20% for validating, and 20% for testing. In the context of the BGL dataset, the ratio mentioned above is 10%, 45%, and 45%, which can be attributed to constraints imposed by limited resources. Also, the Tomek link technique was employed to address the data imbalance. Finally, CoLog is trained on each log dataset and tested on the same dataset for generating results.

Metrics

The performance of CoLog is assessed using multiple metrics, including precision, recall, F1-score, and accuracy. These metrics are computed based on the values of true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN). The scikit-learn library¹⁴² is used to implement metrics with the "macro average" option. Evaluation metrics are computed for each dataset on individual files, and then the average is derived across all files.

Precision: Precision quantifies the proportion of correctly identified positive instances across all positive predictions generated by the CoLog. The precision formula is defined as follows:

$$Precision = \frac{TP}{TP + FP} \quad (18)$$

where TP is correctly predicted positive instances and FP is incorrectly predicted positive instances.

Recall or Sensitivity: Recall quantifies the proportion of actual positive cases correctly detected by the CoLog. The mathematical expression representing the concept of recall is as follows:

$$Recall = \frac{TP}{TP + FN}. \quad (19)$$

where FN is incorrectly predicted negative instances.

F1-score: F1-score pertains to evaluating the balance between precision and recall. The mathematical expression representing the F1-score is as follows:

$$F1 - score = 2 \times \frac{Precision \times Recall}{Precision + Recall} = \frac{2 \times TP}{2 \times TP + FP + FN}. \quad (20)$$

Accuracy: Accuracy quantifies the proportion of correct predictions made by the CoLog. The following equation gives the mathematical expression representing accuracy:

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN}. \quad (21)$$

where TN is correctly predicted negative instances.

Results

In this section, we discuss experiments conducted on the CoLog, whereby several diagrams, including the confusion matrix, normalized confusion matrix, receiver operating characteristic (ROC) curve, and precision-recall (PR) curve, were utilized. The comprehensive CoLog outcomes on publicly available log datasets are presented in Table 3. Additionally, the graphical depictions of these results are illustrated in Figs. 8, 9, 10, 11, 12, 13, 14. Also, subsections are organized as follows. In the subsequent analysis, we compare CoLog and other log anomaly detection approaches by applying them to the above mentioned datasets. Next, we will discuss the outcomes of the hyperparameter tuning process, the impact of the training ratio, the influence of various approaches for addressing the class imbalance, the effect of window size, and the presentation of the vectors derived from the CoLog. Next, let us examine the generalization or robustness of the CoLog in detail. Subsequently, we discuss the significant design components of the CoLog as the ablation study. Next, we undertake groundbreaking research. Subsequently, we will compare our contributions with other log anomaly detection techniques.

The CoLog has exhibited exceptional versatility when applied to diverse datasets, consistently attaining a high level of performance. The method's efficacy is apparent through its high accuracy in classifying all tested datasets, where CoLog achieves near-perfect or perfect classification scores. CoLog demonstrates flawless classification performance on the Casper, Jhuisi, Honey7, and Zookeeper datasets, exhibiting a complete absence of false positives or negatives. Similarly, CoLog exhibited outstanding performance on the Nssal and Hadoop datasets, with few occurrences of false positives and negatives, where all anomalies in the Hadoop dataset are detected. Even when applied to the large BGL dataset, CoLog maintains exceptional performance, accurately

Dataset	Average	Class	# Samples	Precision	Recall	F1-Score	Accuracy
Casper		0: Anomaly	243	100	100	100	100
		1: Normal	1976	100	100	100	
	Macro		2219	100	100	100	
Jhuisi		0: Anomaly	536	100	100	100	100
		1: Normal	1814	100	100	100	
	Macro		2350	100	100	100	
Nssal		0: Anomaly	3147	99.936	99.841	99.889	99.967
		1: Normal	18271	99.972	99.989	99.981	
	Macro		21418	99.955	99.915	99.935	
Honey7		0: Anomaly	110	100	100	100	100
		1: Normal	1633	100	100	100	
	Macro		1743	100	100	100	
Zookeeper		0: Anomaly	9702	100	100	100	100
		1: Normal	5176	100	100	100	
	Macro		14878	100	100	100	
Hadoop		0: Anomaly	2289	100	99.913	99.956	99.994
		1: Normal	33893	99.994	100	99.997	
	Macro		36182	99.997	99.956	99.977	
BlueGene/L		0: Anomaly	156807	99.999	99.980	99.989	99.998
		1: Normal	1964266	99.998	100	99.999	
	Macro		2121073	99.999	99.990	99.994	

Table 3. Comprehensive CoLog outcomes on publicly available log datasets.

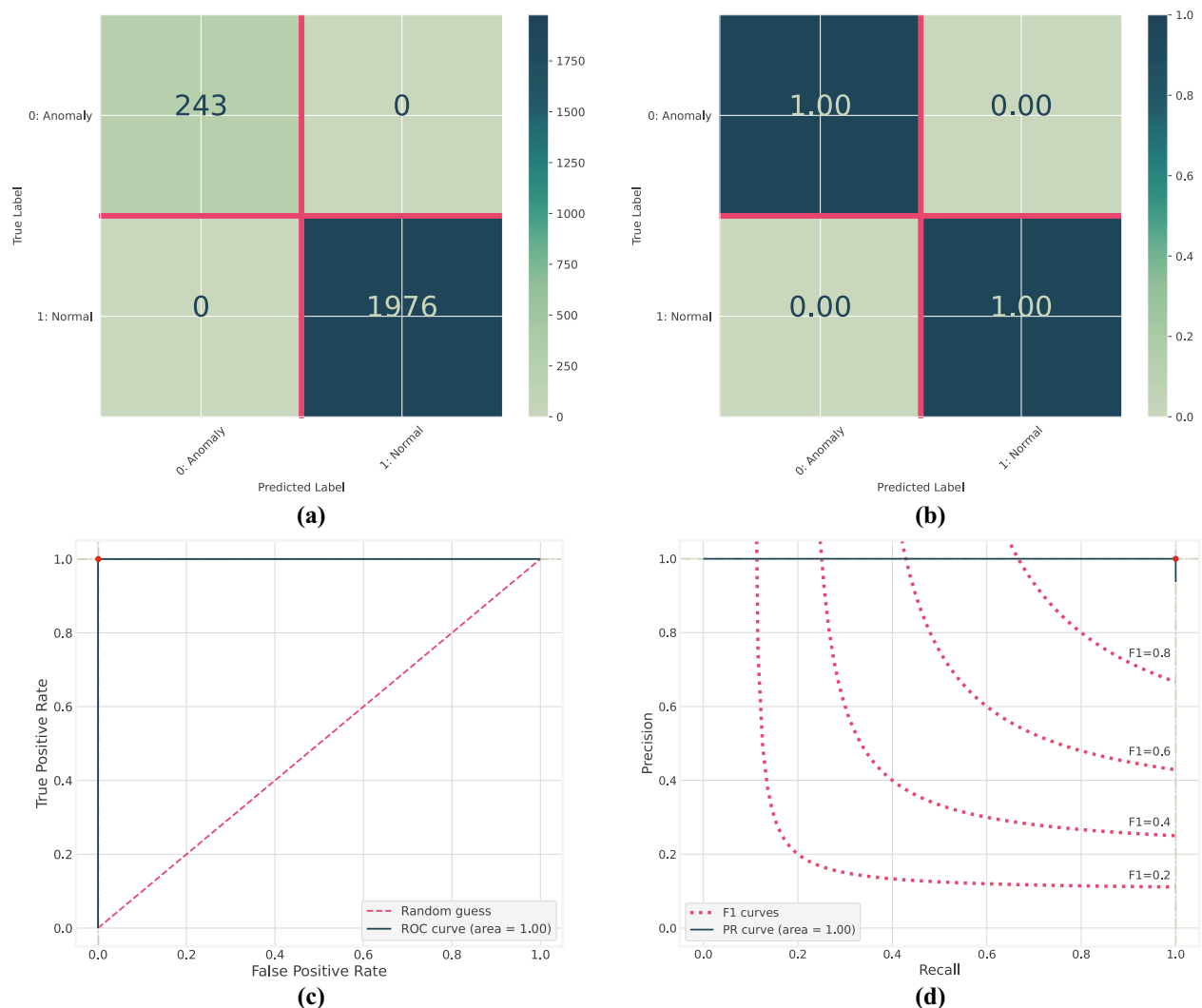


Fig. 8. The collection of output visualizations generated by the CoLog when applied to the Casper log dataset. (a) confusion matrix, (b) normalized confusion matrix, (c) receiver operating characteristics curve, and (d) precision-recall curve.

identifying 1,964,265 true positives and 156,775 true negatives while exhibiting few mistakes with only 1 false negative. The area under the curve (AUC) values for the ROC and PR curves are consistently near 100 or 100 across all datasets, providing additional evidence of the CoLog's strong performance.

The exceptional outcomes of CoLog can be attributed to several significant contributions. Initially, in CoLog, semantic and sequence log modalities collaborate. Utilizing sequential features besides semantic features augments the available knowledge and boosts the model's capacity to identify anomalies. Furthermore, distinct neural networks are used for each modality, which enables CoLog to control the interactions between modalities, yielding more reliable outcomes. Finally, a latent space is used to fuse the modalities. The significance of this matter lies in the fact that various modalities exhibit dissimilar characteristics, necessitating a sophisticated fusion mechanism for their meaningful integration. These contributions demonstrate the adaptability of CoLog and establish its reliability as a tool for detecting log anomalies in various scenarios.

CoLog versus the field: a comparative study

We compared CoLog and some log anomaly detection algorithms available in the [loglizer](#)¹¹¹ and [deeploglizer](#)²⁴ toolboxes. The loglizer is utilized for machine learning algorithms, whereas the deeploglizer is employed for deep learning-based methods. These toolboxes consist of five supervised detection approaches, including the logistic regression (LR)¹⁴³, support vector machines (SVM)¹⁰⁹, and Decision Tree (DT) [109] from the loglizer toolbox, as well as the Attentional BiLSTM [62] and CNN [45] from the deeploglizer toolbox. Furthermore, four unsupervised techniques, including the isolation forest (IF)¹⁴⁴ and principal component analysis (PCA)¹² algorithms from loglizer, the LSTM³⁶ and Transformer⁸⁰ implemented in the deeploglizer. In addition, we evaluate the performance of CoLog in comparison to pylogsentiment. The pylogsentiment is a tool for sentiment

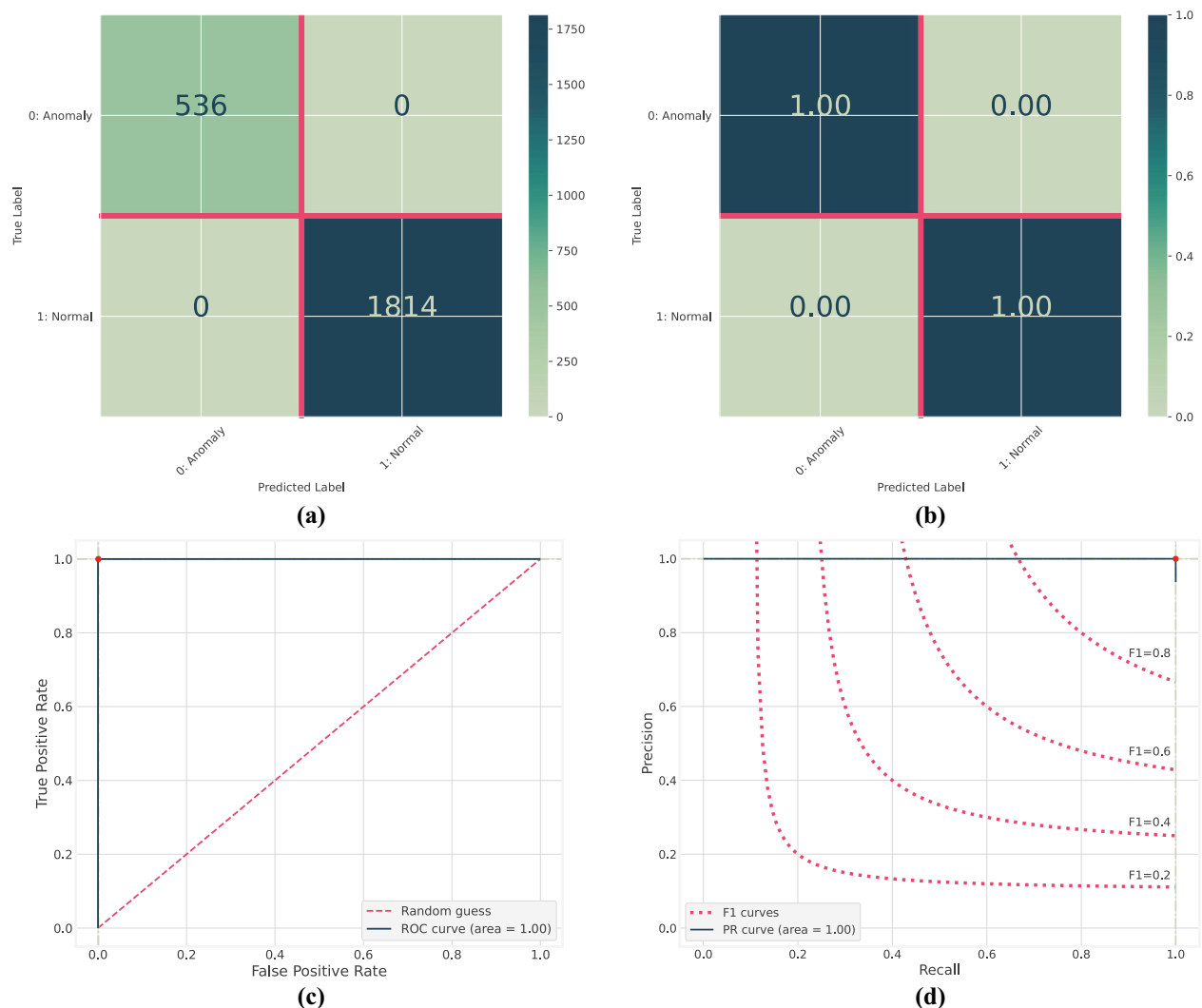


Fig. 9. The collection of output visualizations generated by the CoLog when applied to the Jhuisi log dataset. (a) confusion matrix, (b) normalized confusion matrix, (c) receiver operating characteristics curve, and (d) precision-recall curve.

analysis in OS logs. It uses GRU to identify abnormalities and classify log entries according to their sentiment. This aids in detecting potential issues and evaluating a system's overall state.

Prior to feeding the log datasets into the loglizer or deeploglizer models, we employ the Drain and nerlogparser tools to separate all the log records and extract the log message. The hyperparameters of the models are set to the same configuration as recommended in the loglizer and deeploglizer toolboxes. Furthermore, sliding partitioning is implemented to perform log partitioning. Then, algorithms in loglizer and deeploglizer are employed to process these sequences of log events. Tables 4, 5, 6, 7, 8, 9, 10 present the performance evaluation of CoLog and other anomaly detection methods.

The F1-scores of all techniques are presented at the mean level in Table 11. Of all the methods, pylogsentiment demonstrates superior performance due to its balancing between the two sentiment classes. This results in a more effective deep learning model for reliably identifying anomalous activities within the minority class. The pylogsentiment achieves an average F1-score of 99.135. The performance of other deep learning-based models, including Transformer, CNN, attentional BiLSTM, and LSTM, on all datasets are similar. For instance, they yield similar mean F1-scores of 97.845, 97.812, 97.661, and 96.765, respectively. These methods exhibit enhanced performance due to their utilization of deep learning for anomaly detection. Deep learning is adept at anomaly detection in logs due to its capability to manage unstructured data and autonomously extract features. It effectively handles large data volumes and, when employing log semantics, adapts to evolving abnormalities over time. Transformer produces better results because it uses word-level representations to represent log entries. In Table 11, the IF attained a mean F1-score of 49.372, whereas PCA algorithm recorded a mean F1-score of 47.273. Generally, machine learning methods often struggle with log anomaly detection due to the complexity and unstructured nature of log data, distribution shifts introducing unseen anomalies, and the need for sophisticated

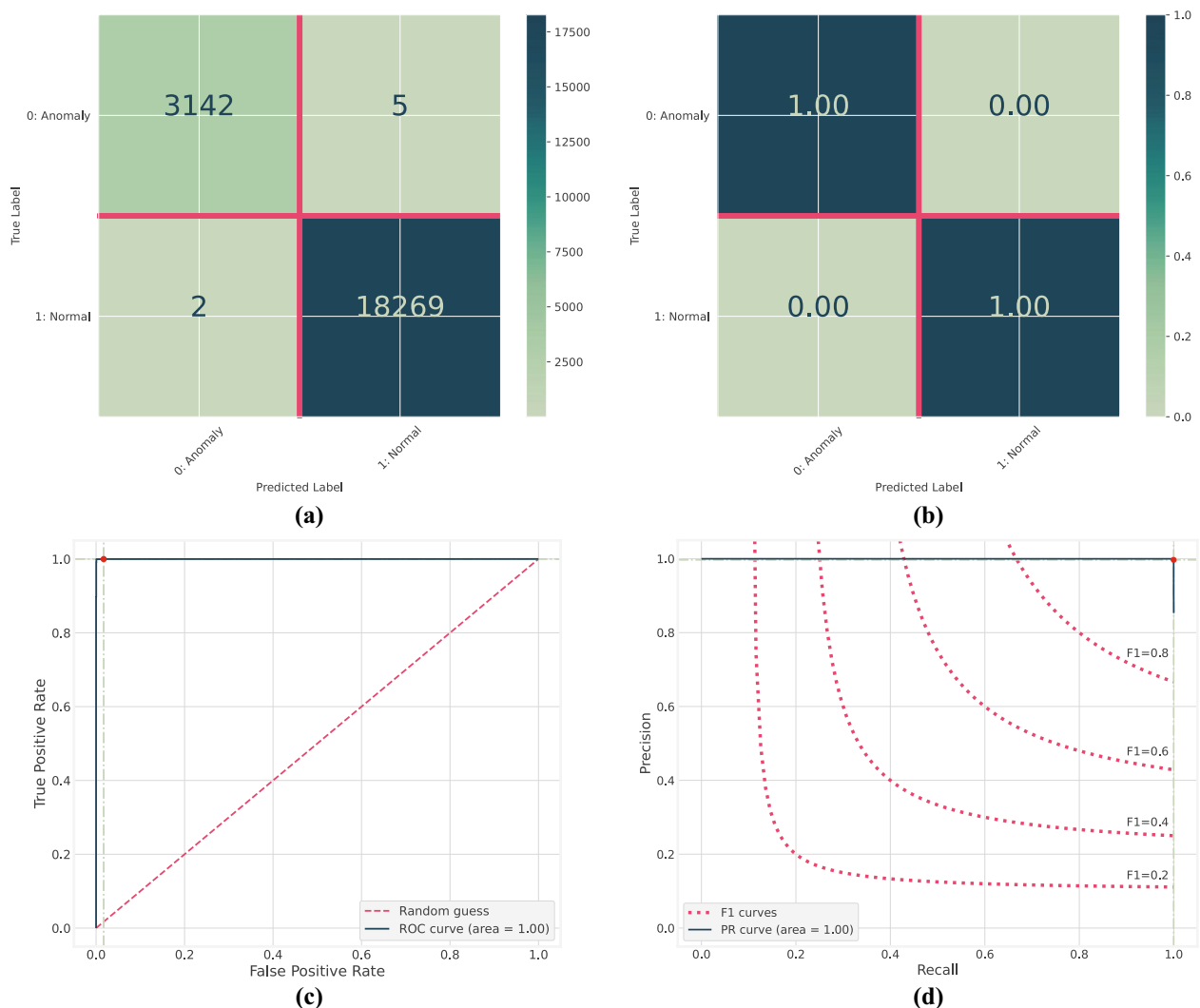


Fig. 10. The collection of output visualizations generated by the CoLog when applied to the Nssal log dataset. (a) confusion matrix, (b) normalized confusion matrix, (c) receiver operating characteristics curve, and (d) precision-recall curve.

feature engineering. Additionally, these models can be sensitive to hyperparameter settings, requiring extensive tuning.

Table 4 to Table 10 indicate that CoLog yields superior performance compared to other techniques across all datasets. Moreover, Table 11 clearly illustrates that CoLog significantly improves anomaly identification. Compared to the second-ranked pylogsentiment, it demonstrates a greater mean F1-score across all datasets. Given that the pylogsentiment has a mean F1-score of 99.135, a further improvement of 0.865 is required to attain perfect anomaly detection on the employed benchmark datasets. CoLog decreases this value to 0.013. This signifies an estimated 98.50% improvement in the available range to enhance the results.

In many cases, unsupervised approaches frequently exhibit inferior performance compared to supervised techniques in log anomaly detection. This is attributable to the absence of labeled data, which limits understanding of complex patterns and reduces accuracy. They also tend to have more false positives. Supervised approaches enhance their accuracy through training on labeled data over time. In this work, we frame log anomaly detection as a multimodal sentiment analysis task, transforming the problem from traditional unsupervised anomaly detection to a supervised classification setting. Conventional anomaly detection methods often treat anomalies as rare or outlier events without explicit labels, which limits the ability of models to learn discriminative features effectively due to the highly imbalanced and variable nature of log data. By interpreting anomalies as negative sentiments within log messages, our approach leverages sentiment analysis techniques to annotate log entries with sentiment labels, thereby enabling supervised learning. This shift allows the model to utilize labeled data for training, improving detection accuracy by capturing subtle semantic patterns and contextual nuances that indicate anomalous system behavior. Consequently, our multimodal sentiment analysis framework benefits from both the rich semantic representation of logs and the robust classification capabilities of supervised learning models, leading to enhanced detection performance compared to purely unsupervised approaches.

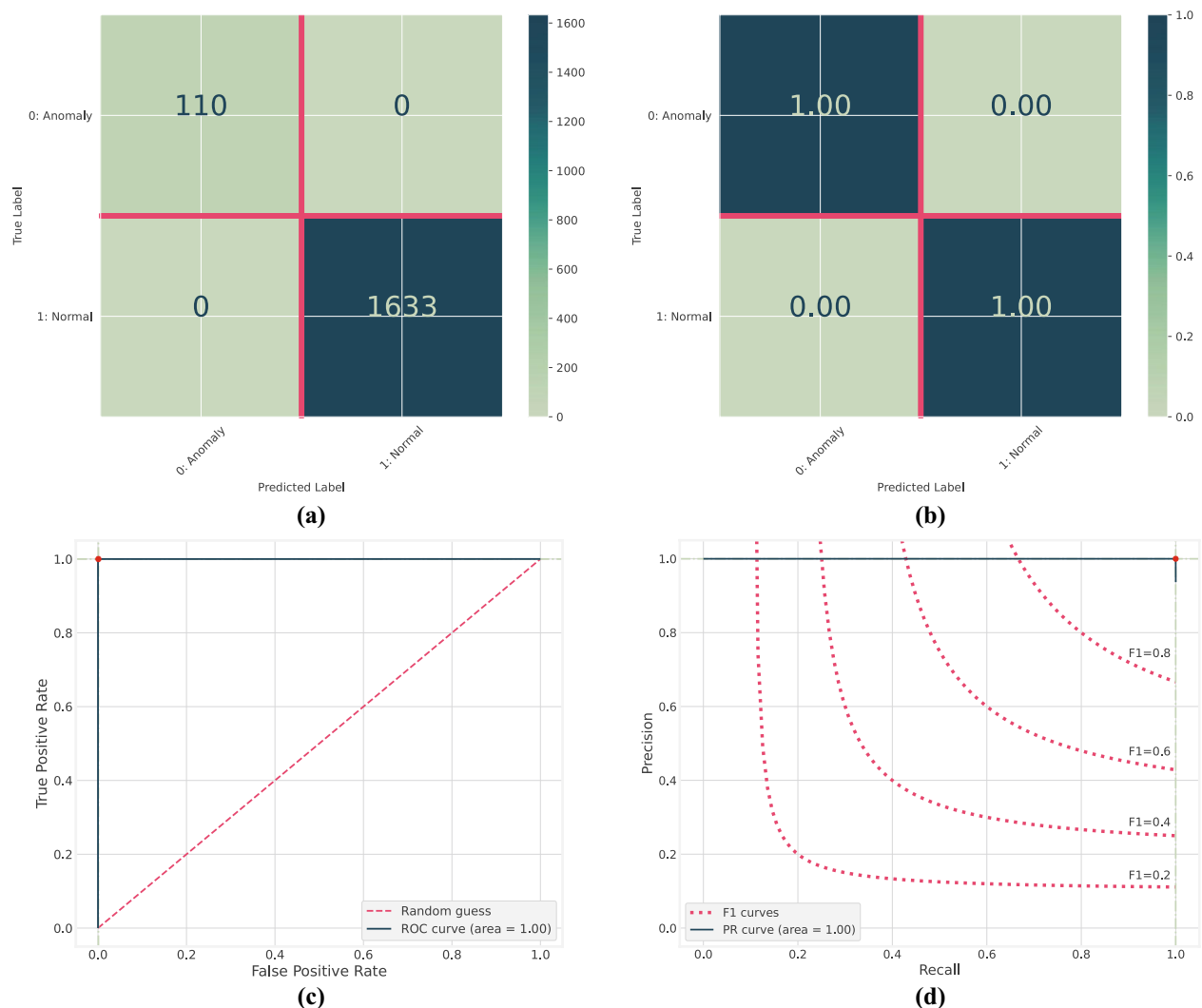


Fig. 11. The collection of output visualizations generated by the CoLog when applied to the Honey7 log dataset. (a) confusion matrix, (b) normalized confusion matrix, (c) receiver operating characteristics curve, and (d) precision-recall curve.

Additionally, CoLog's exceptional efficacy in anomaly detection is due to its novel methodology for learning log data via both sequence and semantic modalities. In contrast to conventional approaches that may focus on a single facet of log data, CoLog incorporates multiple modalities to produce a more comprehensive representation of log data. This dual-modality learning enables CoLog to gain a more thorough understanding of log data that might otherwise be overlooked. Another significant component enhancing CoLog's efficacy is its collaborative approach to acquiring log representations. Unlike previous approaches that may analyze logs in isolation or with minimal interaction across various data modalities, CoLog guarantees the simultaneous learning of sequence and semantic information. This collaborative method yields more comprehensive and refined representations of log data, thereby enhancing the model's ability to distinguish between normal and abnormal activity. Jointly examining logs facilitates enhanced adaptability to various log contexts and scenarios. This versatility is essential in practical applications where logs may differ markedly in form and content. In addition, in many traditional methods, the integration of various data modalities can lead to noise and inconsistencies, ultimately degrading performance. One of the standout features of CoLog is its modality adaptation layer, which plays a crucial role in refining its performance by removing impurities that may arise from combining different modalities. This layer acts as a filter, ensuring that only relevant and complementary features are fused. By eliminating conflicting or redundant information, the MAL prevents the degradation of performance that could occur if modalities were combined without such careful consideration. CoLog's ability to filter out these impurities ensures that the resulting log representations are cleaner and more reliable, enhancing the overall accuracy of the analysis. As a result, CoLog is better equipped to handle the variability and complexity inherent in log data, making it a more effective tool for administrators in the field. CoLog also utilizes a latent space for meaningfully fusing different modalities of log data. Instead of simply concatenating features from each modality or processing them separately, CoLog explores a shared latent space where these features can be integrated effectively. By mapping

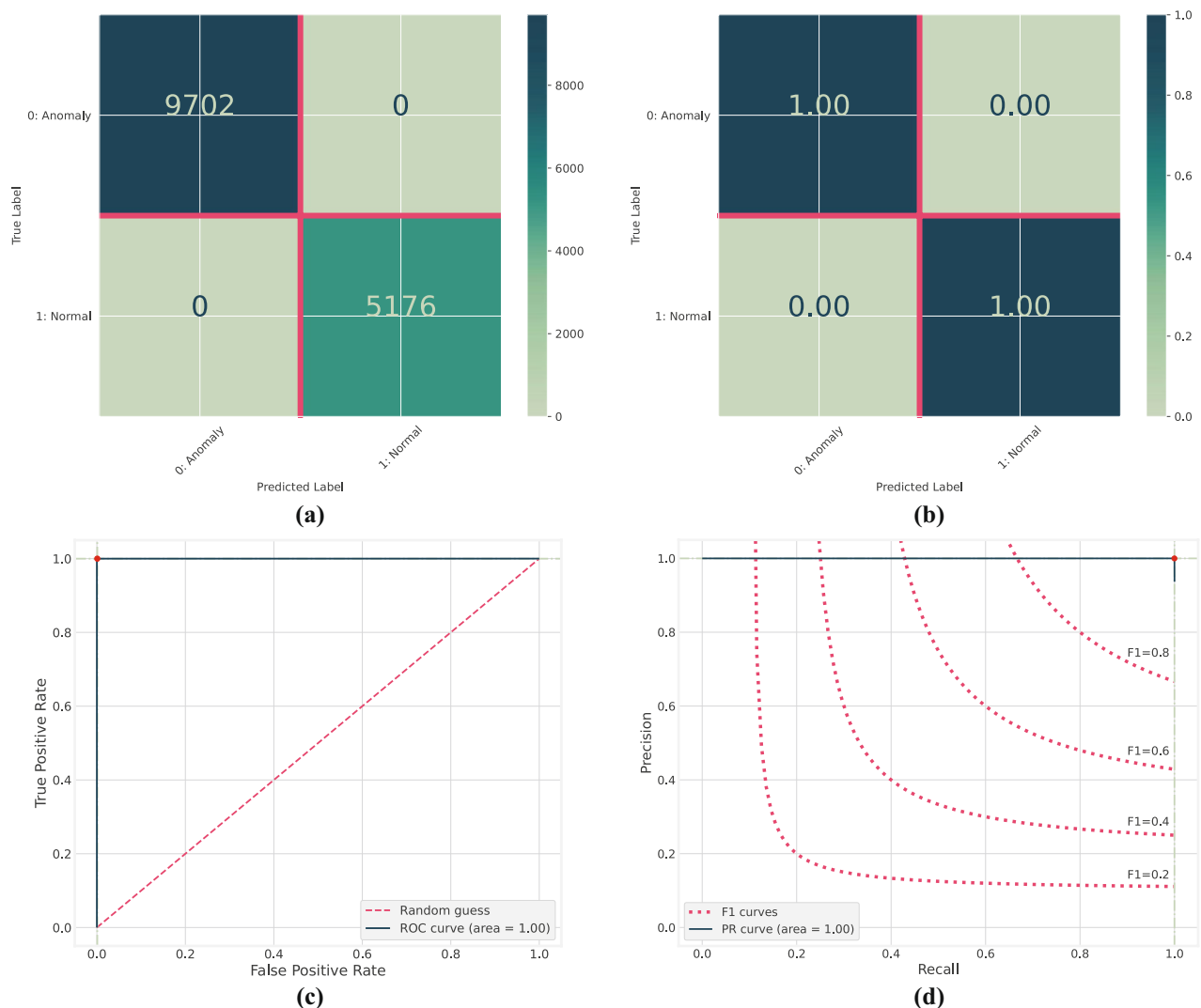


Fig. 12. The output collection of visualizations generated by the CoLog when applied to the Zookeeper dataset. (a) confusion matrix, (b) normalized confusion matrix, (c) receiver operating characteristics curve, and (d) precision-recall curve.

log data into a common latent space, CoLog can capture complex relationships and dependencies that are essential for accurate anomaly detection.

Table 12 evaluates the efficiency of different models by recording the time spent on training and testing processes on the Hadoop dataset. The LSTM, Attentional BiLSTM, CNN, and Transformer methodologies necessitate considerably less time for training and inference compared to pylogsentiment and CoLog. This results from utilizing log keys rather than the semantic features of logs, which require far fewer computer resources. CoLog requires less training time than pylogsentiment, although it is inferior during the inference phase.

Enhancing success: study on hyperparameters tuning

Hyperparameters tuning is essential in deep learning as it directly influences model performance and generalization¹⁴⁵. This process can substantially improve model accuracy and efficiency¹⁴⁶. Appropriate tuning addresses challenges such as overfitting or underfitting, which is essential in noisy and mutation-prone log data. Furthermore, it facilitates the examination of various model configurations, resulting in more resilient and dependable results. Considering the importance of tuning, specifically in log anomaly detection, we employ the Ray package in Python to tune CoLog. In the tuning procedure, we utilize three datasets, Casper, Jhuisei, and Honey7, and apply 324 distinct configurations of the CoLog on them. Based on the tuning results of the Casper, Jhuisei, and Honey7 datasets, the hyperparameters selection criterion for each dataset is to achieve optimal accuracy in the shortest time. The overall optimal configuration is the resultant of all datasets' optimum configurations. Due to page constraints, this paper only covers the 27 most efficient runs out of 108 runs for each dataset (Complete tuning results are available on CoLog's GitHub repository.). The chosen hyperparameters are detailed in Section "Hyperparameters".

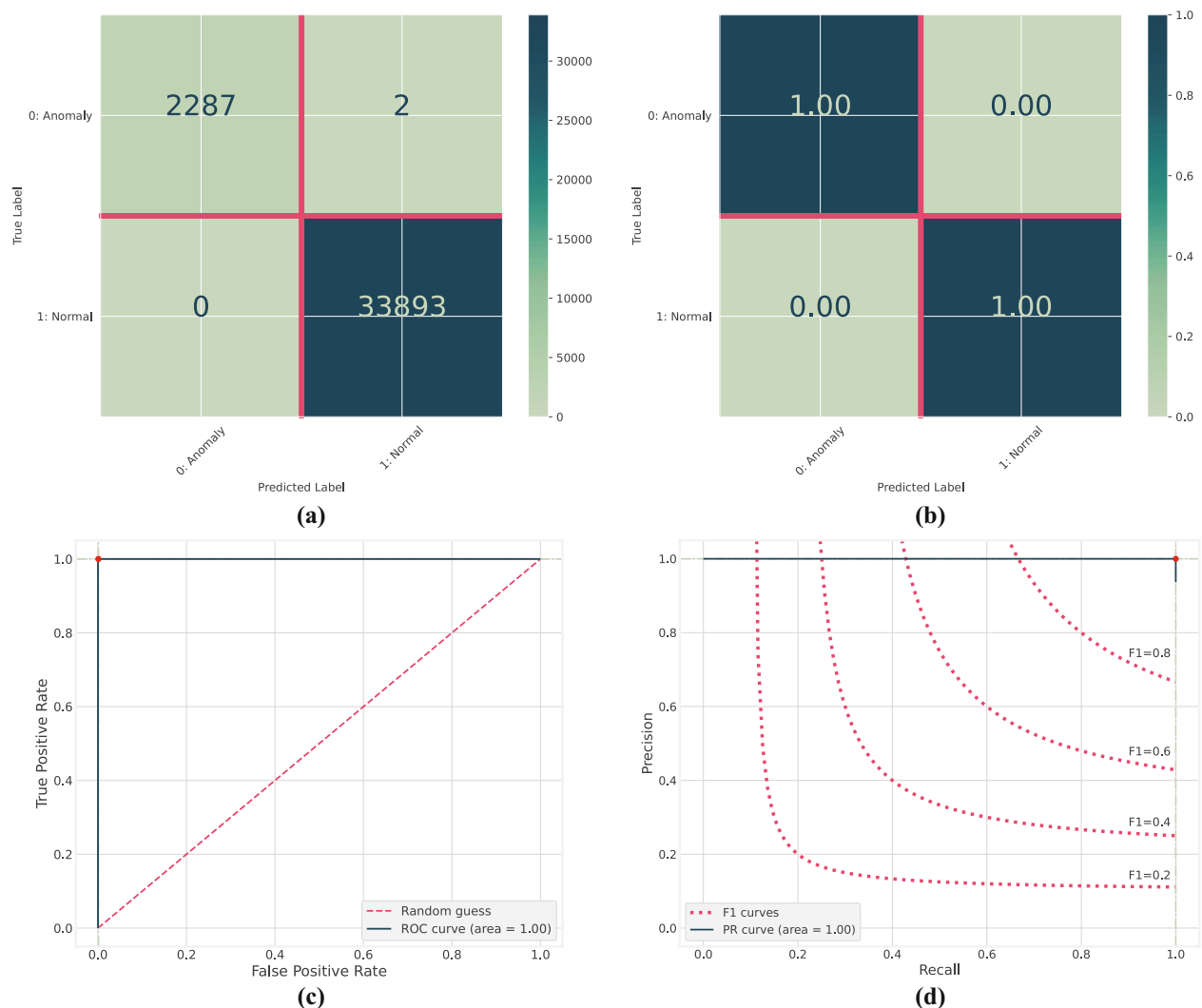


Fig. 13. The collection of output visualizations generated by the CoLog when applied to the Hadoop log dataset. (a) confusion matrix, (b) normalized confusion matrix, (c) receiver operating characteristics curve, and (d) precision-recall curve.

Figure 15 illustrates the results of the top 27 executions of CoLog on the Casper, Jhuisi, and Honey7 datasets, arranged from left to right. The light green bars signify the model's accuracy in that configuration of CoLog, while the rose bars represent the total time required to attain that accuracy. Accordingly, configurations 3384, 6587, and 2868 have achieved optimal outcomes in the shortest time for Casper, Jhuisi, and Honey7 datasets, respectively. Figure 15 also illustrates the tag for the optimum configurations of each dataset.

Consequences of ratio: examination of train ratio impact

Inadequate and imbalanced training data, such as log data, can result in overfitting, wherein the model is talented in training data but fails on novel data¹⁴⁷. Furthermore, models trained with a suitable train-test ratio generally exhibit superior performance¹⁴⁸. Consequently, determining an optimal train-test data ratio is essential. The percentage of training data influences the model's robustness to noise and data mutation, which are abundantly seen in the log data. An optimally selected ratio enables the model to learn underlying patterns in the data while maintaining robustness against outliers and noise. This is particularly vital in practical applications, for example, log-based anomaly detection, where data may be noisy and unreliable. We assess the influence of various train ratios by applying CoLog on the Casper, Jhuisi, and Honey7 datasets, which have been preprocessed with train ratios of 0.4, 0.5, 0.6, 0.7, 0.8, and 0.9. Figure 16 illustrates the outcomes of analyzing various training data rates. The optimum result occurs when CoLog is applied to the Jhuisi and Honey7 datasets with a training ratio of 0.6. The optimal ratio for the Casper dataset is 0.8. Based on the above mentioned ratios, the F1-score is 100 for the Casper dataset, 99.879 for the Jhuisi dataset, and 100 for the Honey7 dataset. Finding a proper training ratio is crucial in log anomaly detection. The ideal proportion of training data substantially affects the convergence time of deep learning models, which is vital in this context. A balanced ratio guarantees the model obtains diverse and

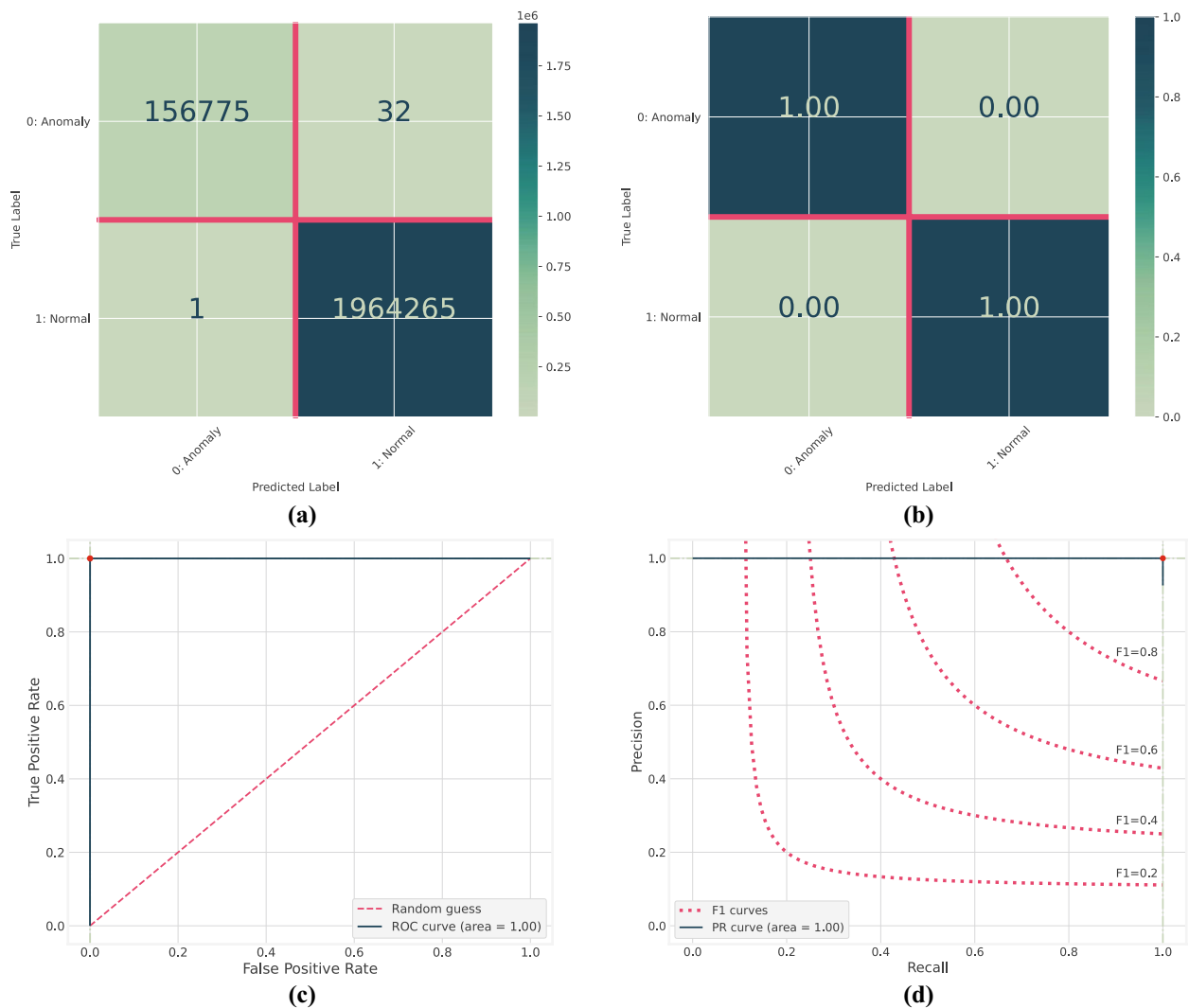


Fig. 14. The collection of output visualizations generated by the CoLog when applied to the BlueGene/L log dataset. (a) confusion matrix, (b) normalized confusion matrix, (c) receiver operating characteristics curve, and (d) precision-recall curve.

Anomaly Detection Technique	Precision	Recall	F1-Score	Accuracy
Supervised Methods				
Logistic Regression ¹⁴³	66.959	60.863	59.700	90.993
Support Vector Machines ¹⁰⁹	58.614	60.757	59.482	90.967
Decision Tree ¹¹⁰	83.466	77.037	79.488	94.998
Attentional BiLSTM ⁶³	99.766	99.872	99.819	99.834
Convolutional Neural Network ⁴⁶	99.766	99.872	99.819	99.834
pylogsentiment ³⁸	99.487	99.413	99.449	99.459
Unsupervised Methods				
Isolation Forest ¹⁴⁴	52.407	50.650	49.926	88.149
Principal Component Analysis ¹²	51.205	50.282	49.362	87.480
LSTM ³⁶	97.973	98.843	98.380	98.505
Transformer ⁸⁰	98.409	99.100	98.738	98.837
CoLog ¹	100	100	100	100

Table 4. CoLog in comparison to other log anomaly detection methods on the Casper dataset. Bold numbers indicate the outstanding results. CoLog is a supervised method.

Anomaly detection technique	Precision	Recall	F1-Score	Accuracy
Supervised Methods				
Logistic Regression ¹⁴³	68.373	66.127	64.886	79.182
Support Vector Machines ¹⁰⁹	63.643	66.506	64.051	80.914
Decision Tree ¹¹⁰	91.534	89.769	90.550	93.313
Attentional BiLSTM ⁶³	95.123	97.270	96.169	99.341
Convolutional Neural Network ⁴⁶	97.139	94.885	95.982	99.341
pylogsentiment ³⁸	98.867	98.761	98.813	98.850
Unsupervised Methods				
Isolation Forest ¹⁴⁴	57.519	52.774	51.700	74.707
Principal Component Analysis ¹²	44.828	49.299	45.014	75.448
LSTM ³⁶	96.879	92.385	94.508	99.121
Transformer ⁸⁰	96.879	92.385	94.508	99.121
CoLog ¹	100	100	100	100

Table 5. CoLog in comparison to other log anomaly detection methods on the Jhuisi dataset. Bold numbers indicate the outstanding results. CoLog is a supervised method.

Anomaly detection technique	Precision	Recall	F1-Score	Accuracy
Supervised Methods				
Logistic Regression ¹⁴³	85.133	74.728	76.476	97.604
Support Vector Machines ¹⁰⁹	80.206	74.935	76.474	97.655
Decision Tree ¹¹⁰	94.791	87.700	89.470	98.063
Attentional BiLSTM ⁶³	96.750	98.805	97.754	99.813
Convolutional Neural Network ⁴⁶	96.703	98.243	97.460	99.789
pylogsentiment ³⁸	97.170	96.050	96.602	99.020
Unsupervised Methods				
Isolation Forest ¹⁴⁴	65.504	57.352	56.101	80.967
Principal Component Analysis ¹²	52.642	53.827	49.505	80.614
LSTM ³⁶	96.148	97.669	96.896	99.742
Transformer ⁸⁰	96.304	99.354	97.778	99.813
CoLog ¹	99.955	99.915	99.935	99.967

Table 6. CoLog in comparison to other log anomaly detection methods on the Nssal dataset. Bold numbers indicate the outstanding results. CoLog is a supervised method.

Anomaly detection technique	Precision	Recall	F1-Score	Accuracy
Supervised Methods				
Logistic Regression ¹⁴³	68.143	70.000	69.042	96.286
Support Vector Machines ¹⁰⁹	68.143	70.000	69.042	96.286
Decision Tree ¹¹⁰	93.260	83.307	86.359	97.471
Attentional BiLSTM ⁶³	100	100	100	100
Convolutional Neural Network ⁴⁶	100	100	100	100
pylogsentiment ³⁸	99.970	99.107	99.535	99.943
Unsupervised Methods				
Isolation Forest ¹⁴⁴	47.509	50.948	48.064	85.966
Principal Component Analysis ¹²	60.871	58.898	55.943	88.279
LSTM ³⁶	96.212	98.810	97.429	98.155
Transformer ⁸⁰	96.923	99.048	97.932	98.524
CoLog ¹	100	100	100	100

Table 7. CoLog in comparison to other log anomaly detection methods on the Honey7 dataset. Bold numbers indicate the outstanding results. CoLog is a supervised method.

Anomaly detection technique	Precision	Recall	F1-Score	Accuracy
Supervised Methods				
Logistic Regression ¹⁴³	98.369	98.464	98.416	98.562
Support Vector Machines ¹⁰⁹	97.987	97.769	97.877	98.078
Decision Tree ¹¹⁰	98.599	98.880	98.737	98.851
Attentional BiLSTM ⁶³	95.783	92.928	94.300	98.387
Convolutional Neural Network ⁴⁶	95.121	92.662	93.850	98.252
pylogsentiment ³⁸	99.722	99.898	99.810	99.973
Unsupervised Methods				
Isolation Forest ¹⁴⁴	32.607	50.000	39.473	65.215
Principal Component Analysis ¹²	79.011	51.209	42.121	66.021
LSTM ³⁶	95.825	91.887	93.750	98.252
Transformer ⁸⁰	99.890	99.416	99.652	99.361
CoLog	100	100	100	100

Table 8. CoLog in comparison to other log anomaly detection methods on the Zookeeper dataset. Bold numbers indicate the outstanding results. CoLog is a supervised method.

Anomaly detection technique	Precision	Recall	F1-Score	Accuracy
Supervised Methods				
Logistic Regression ¹⁴³	48.523	50.000	49.250	97.046
Support Vector Machines ¹⁰⁹	48.523	50.000	49.250	97.046
Decision Tree ¹¹⁰	48.523	50.000	49.250	97.046
Attentional BiLSTM ⁶³	97.640	97.955	97.792	97.902
Convolutional Neural Network ⁴⁶	99.719	99.847	99.783	99.955
pylogsentiment ³⁸	99.886	99.732	99.809	99.905
Unsupervised Methods				
Isolation Forest ¹⁴⁴	47.702	50.000	48.824	54.034
Principal Component Analysis ¹²	49.995	49.996	49.996	58.214
LSTM ³⁶	99.850	97.397	98.589	99.715
Transformer ⁸⁰	97.280	99.833	98.518	99.685
CoLog	99.997	99.956	99.977	99.994

Table 9. CoLog in comparison to other log anomaly detection methods on the Hadoop dataset. Bold numbers indicate the outstanding results. CoLog is a supervised method.

Anomaly detection technique	Precision	Recall	F1-Score	Accuracy
Supervised Methods				
Logistic Regression ¹⁴³	54.028	51.852	52.092	90.368
Support Vector Machines ¹⁰⁹	46.314	50.000	48.087	92.628
Decision Tree ¹¹⁰	60.576	50.998	50.303	92.348
Attentional BiLSTM ⁶³	97.640	97.955	97.792	97.902
Convolutional Neural Network ⁴⁶	97.640	97.955	97.792	97.902
pylogsentiment ³⁸	99.892	99.963	99.928	99.980
Unsupervised Methods				
Isolation Forest ¹⁴⁴	53.081	50.047	51.519	47.389
Principal Component Analysis ¹²	51.168	54.260	38.970	48.487
LSTM ³⁶	97.414	98.296	97.806	97.902
Transformer ⁸⁰	97.640	97.955	97.792	97.902
CoLog ¹	99.999	99.990	99.994	99.998

Table 10. CoLog in comparison to other log anomaly detection methods on the BlueGene/L dataset. Bold numbers indicate the outstanding results. CoLog is a supervised method.

Anomaly detection technique	Mean F1-score
Principal Component Analysis ¹²	47.273
Isolation Forest ¹⁴⁴	49.372
Support Vector Machines ¹⁰⁹	66.323
Logistic Regression ¹⁴³	67.123
Decision Tree ¹¹⁰	77.737
LSTM ³⁶	96.765
Attentional BiLSTM ⁶³	97.661
Convolutional Neural Network ⁴⁶	97.812
Transformer ⁸⁰	97.845
pylogsentiment ³⁸	99.135
CoLog	99.987

Table 11. Ranking CoLog and other log anomaly detection methods. Bold numbers indicate the outstanding results.

Anomaly detection technique	Training time (Seconds)		Inference time (Seconds)	
	Overall	Per-sample	Overall	Per-sample
LSTM ³⁶	7.72	0.00112	0.14	0.00002
Attentional BiLSTM ⁶³	25.71	0.00372	0.21	0.00003
Convolutional Neural Network ⁴⁶	14.85	0.00215	0.07	0.00001
Transformer ⁸⁰	9.05	0.00131	0.09	0.00001
pylogsentiment ³⁸	2615.29	0.01138	94.78	0.00262
CoLog	2479.02	0.01079	124.00	0.00343

Table 12. Efficiency of deep learning models on Hadoop Dataset.

representative samples during training, accelerating the learning process. An unbalanced ratio may result in slow convergence or cause it to become bound in local minima.

Balancing endeavor: comparative study on class imbalance solving methods

Data imbalance plays a crucial role in deep learning, profoundly affecting model performance and generalization. Imbalanced training data causes models to exhibit bias towards the majority class, resulting in suboptimal performance on the minority class^{149,150}. This poses significant challenges in automated log anomaly detection, where the minority class frequently signifies critical instances³⁸. Mitigating data imbalance enables deep learning models to generate precise predictions across all imbalanced classes, augmenting their practical applicability in real-world scenarios. Numerous approaches have been devised to alleviate the impact of data imbalance, focusing on either equalizing class distribution or modifying the learning process to prioritize the minority class^{126–131}.

CoLog employs under-sampling techniques due to the similarity among numerous log entries in the majority class. We implement class imbalance solving across all datasets with the Tomek link approach. Figure 17 illustrates the efficacy of the Tomek link relative to various class balancing techniques, including the condensed nearest neighbor rule¹⁵¹, NearMiss¹²⁸, neighborhood cleaning rule¹⁵², one-sided selection¹⁵³, and random under-sampling across the Casper, Jhuisi, and Honey7 datasets.

Figure 17 illustrates that the Tomek link exhibited the highest mean values across all datasets for precision, recall, F1-score, and accuracy, achieving 99.908%, 99.972%, 99.940%, and 99.957%, respectively. In the Honey7 dataset, the outcomes of the Tomek link method surpass slightly those of alternative techniques. However, the condensed nearest neighbor rule and NearMiss algorithms exhibit poor performance on this dataset. Except for NearMiss, all approaches yield successful results on the Jhuisi dataset. The Tomek link and neighborhood cleaning rule techniques also produce the same results in the Jhuisi dataset. Note that the Tomek link approach attains flawless results of 100% in the Casper dataset.

Marvels of window sizes: study on the effect of window size

We utilize window size 1 with a context sequence type across all datasets. Table 13 to Table 15 illustrate the performance of the context-based window size 1 in contrast to various window sizes and sequence types on the Casper, Jhuisi, and Honey7 datasets.

As shown in Tables 11 to 13, among all the datasets evaluated with varying window sizes, the context-based window size 1 yields the most optimal results at the lowest cost. In the Casper (i.e. Table 13) dataset, all window sizes yield acceptable yet same results, except the background-based window size of 6 and the context-based window sizes of 1, 2, and 12. The above mentioned exceptions attain a 100% result in the evaluation metrics. In



Fig. 15. Collection of graphical illustrations of hyperparameters tuning practices performed by the CoLog on the (a) Casper, (b) Jhuisei, and (c) Honey7 datasets. The criterion for selecting the optimal configuration is to achieve the highest accuracy in the shortest time. Based on this criterion, the results are sorted from left to right. The optimal configuration is also depicted in the respective diagram for each dataset.

the Jhuisei dataset (i.e. Table 14), all results are independently significant, with optimal outcomes achieved with a context-based window size of 1. For Honey7 dataset (i.e. Table 15), background-based window sizes of 2, 3, 6, and 9 yield 100% among all metrics, while context-based window sizes of 1 and 3 also obtain 100%. The rest configurations attain the same results of 99.97%, 99.55%, 99.76%, and 99.94% for precision, recall, F1-score, and accuracy, respectively.

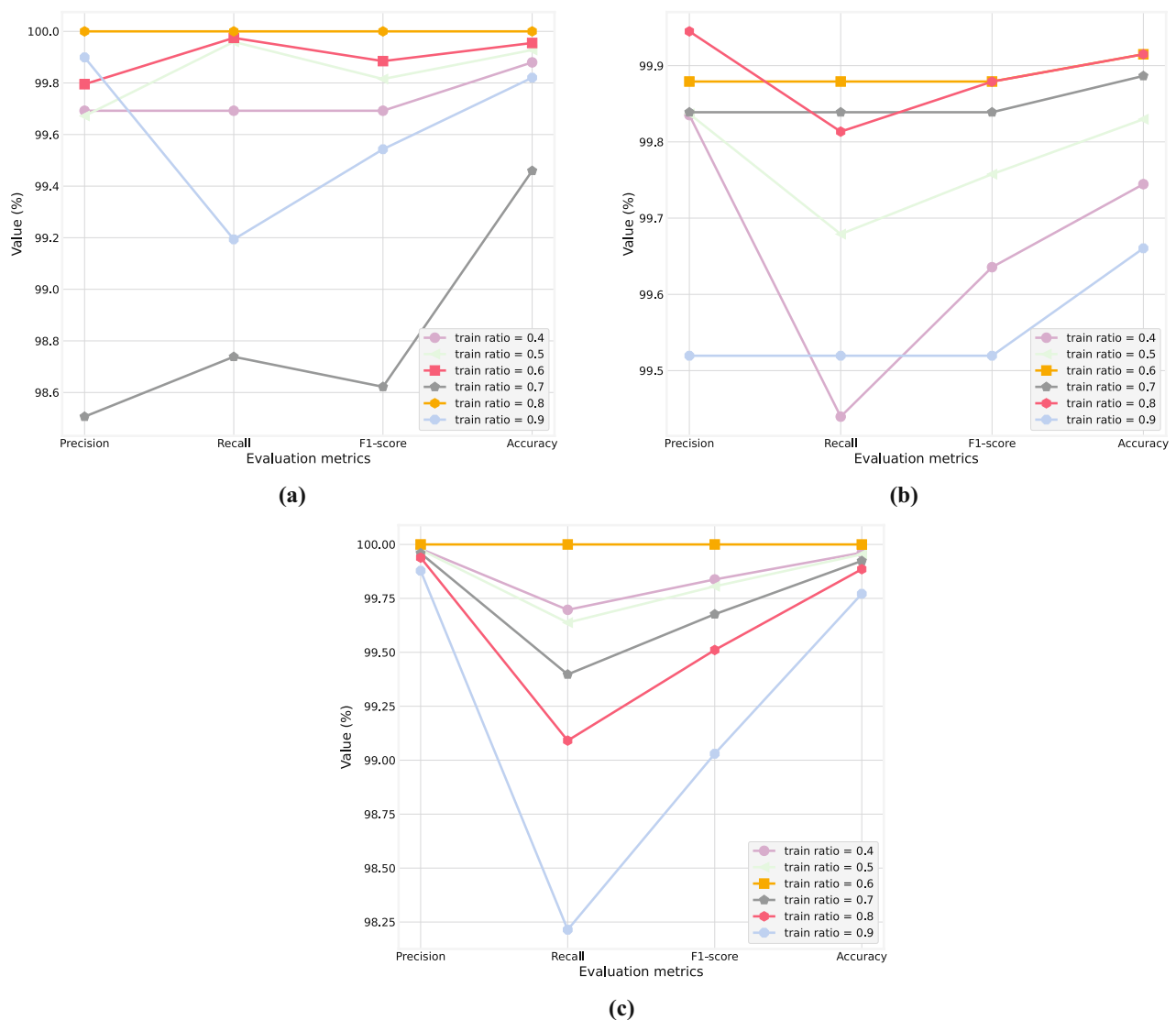


Fig. 16. Visual illustrations of various train-test configurations performed by the CoLog on the (a) Casper, (b) Jhuisi, and (c) Honey7 datasets.

Log landscapes: visualization of CoLog's output vectors

In this experiment, we collect the learned vector representations of log messages from the pre-trained CoLog. Figure 18 illustrates the log vector representations on the Casper, Jhuisi, and Honey7 datasets, demonstrating their lower-dimensional representation of the test splits through the PCA dimensionality reduction technique. We prove that the log vector representations are classified accurately. The normal samples are concentrated closely together. Most anomalies are distributed further than normal samples and clustered together, where there is less error. Consequently, optimal performance could be achieved by applying an argmax prediction function.

Resilience: study on generalization and robustness

We look at the robustness and ability to be generalized of CoLog in identifying unknown abnormalities in previously untrained datasets. The datasets comprise Spark, Honey5, and Windows logs. Nevertheless, CoLog continues to demonstrate excellent results in identifying unknown abnormalities in addition to known anomalies, as demonstrated in Table 16. CoLog attained F1-scores of 98.83% and 99.12% on the Honey5 and Windows datasets, respectively. CoLog has experienced training on several datasets and is capable of addressing unidentified abnormalities. There are three primary interpretations. Initially, we employ word2vec word embeddings to represent each log message's words. word2vec effectively captures the meanings of similar words, such as "warning" and "stopping". Secondly, we utilize sentence-transformers to derive analogous sentence embeddings for similar log messages. Third, we employ a supervised MSA model in a collaborative manner that attains superior performance on both known and novel datasets.

Furthermore, we perform robustness experiments on the Spark Dataset. We train and evaluate CoLog using the original Spark dataset, specifically the training set comprising 10% and the validation set including 45%

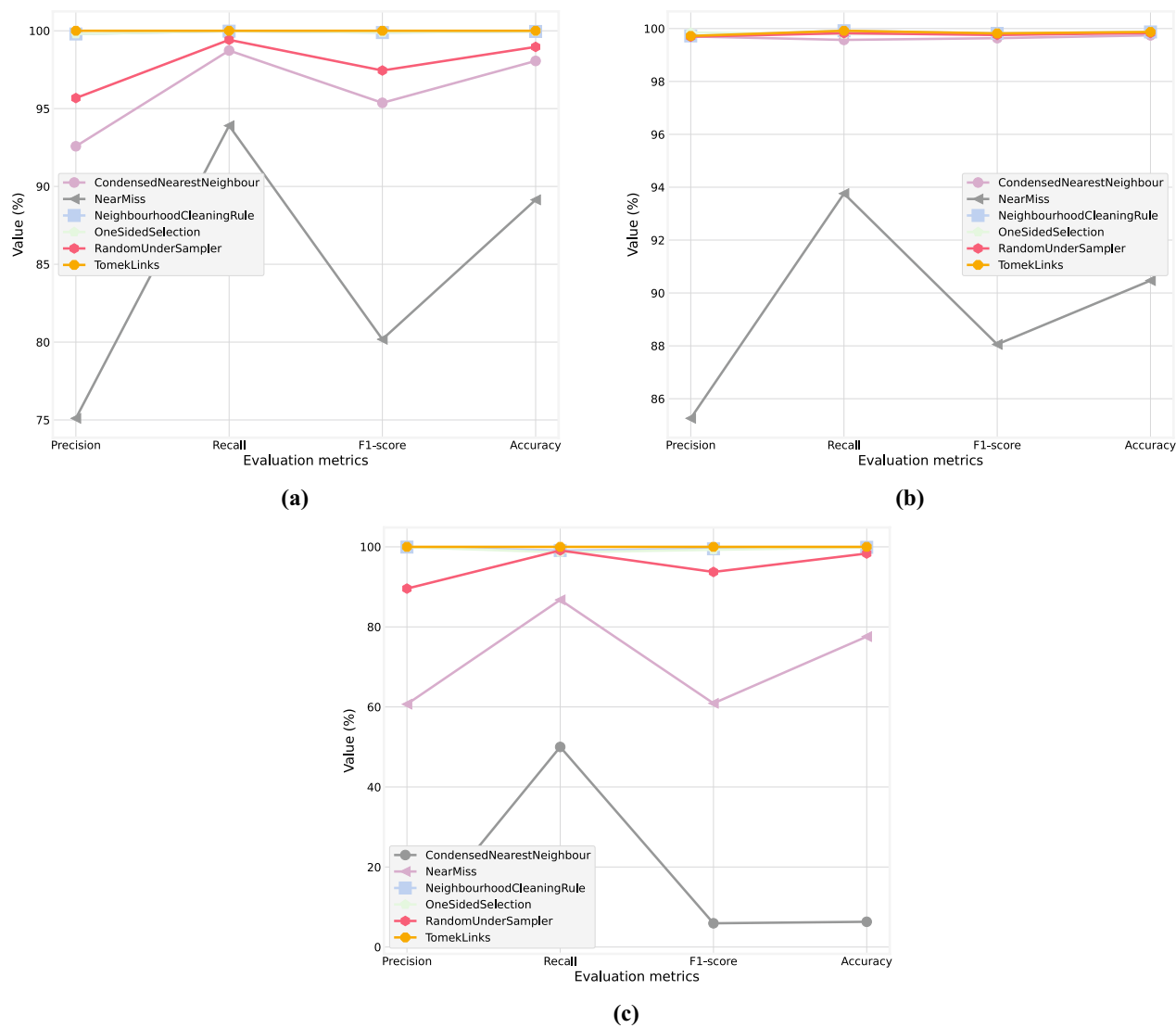


Fig. 17. Comparison of Tomek link with other class imbalance solving methods performed by the CoLog on the (a) Casper, (b) Jhuisi, and (c) Honey7 datasets.

Window size	Sequence type									
	Background					Context				
	1	2	3	6	9	1	2	3	6	9
Precision	99.80	99.80	99.80	100	99.80	100	100	99.80	99.80	99.80
Recall	99.97	99.97	99.97	100	99.97	100	100	99.97	99.97	99.97
F1-Score	99.88	99.88	99.88	100	99.88	100	100	99.88	99.88	99.88
Accuracy	99.95	99.95	99.95	100	99.95	100	100	99.95	99.95	99.95

Table 13. Comparative analysis of CoLog performance across various window sizes and sequence types within the Casper dataset. Bold numbers indicate the outstanding results.

of the total data. Subsequently, we test the trained model on the remaining 45% of the dataset. This synthetic dataset incorporates new log events injected at ratios of 0%, 5%, 10%, 15%, and 20% from alternative datasets. The results of the experiment are demonstrated in Fig. 19. CoLog shows commendable recall metrics. Despite the rising injection ratio of unstable log events, CoLog consistently achieves a high recall rate even at elevated injection levels. For instance, CoLog can maintain a recall of 98.51% at an injection ratio of 20%. It validates that our methodology is adequately adaptable to unstable log events. CoLog interprets all log events as semantic

Window size	Sequence type									
	Background					Context				
	1	2	3	6	9	1	2	3	6	9
Precision	99.97	99.76	99.94	99.88	99.78	100	99.92	99.52	99.85	99.75
Recall	99.91	99.76	99.81	99.88	99.25	100	99.72	99.52	99.79	99.16
F1-Score	99.94	99.76	99.88	99.88	99.51	100	99.82	99.52	99.82	99.45
Accuracy	99.96	99.83	99.91	99.91	99.66	100	99.87	99.66	99.87	99.62

Table 14. Comparative analysis of CoLog performance across various window sizes and sequence types within the Jhuisi dataset. Bold numbers indicate the outstanding results.

Window size	Sequence type									
	Background					Context				
	1	2	3	6	9	1	2	3	6	9
Precision	99.97	100	100	100	100	100	99.97	100	99.97	99.97
Recall	99.55	100	100	100	100	100	99.55	100	99.55	99.55
F1-Score	99.76	100	100	100	100	100	99.76	100	99.76	99.76
Accuracy	99.94	100	100	100	100	100	99.94	100	99.94	99.94

Table 15. Comparative analysis of CoLog performance across various window sizes and sequence types within the Honey7 dataset. Bold numbers indicate the outstanding results.

vectors, enabling it to detect unstable log events with similar semantic meanings. As a result, CoLog can efficiently analyze new log events and could keep on operating on the dataset containing unseen or unstable log events.

Unraveling the impact: ablation study

An ablation study was performed on the Casper, Jhuisi, and Honey7 datasets, with outcomes summarized in Table 17.

To conduct an ablation study, we eliminate sequence modality from multimodal information to assess the impact of background or context of events on model performance. The elimination of sequence modality results in reduced performance, which signifies that sequential signals are essential for addressing log-based anomaly detection and illustrate the complementarity between log events and the background or context of events. Additionally, we eliminate module MHIA from CoLog and update it with MHA. Furthermore, we eliminate MAL and the balancing layer, which was intended to enable the fusion of various modalities, resulting in a decrease in all performance metrics. These results demonstrate the efficacy of MHIA, MAL, and the balancing layer in representation learning for detecting anomalies in logs. Finally, evaluate the distinct interconnections among diverse modules. The results demonstrate that CoLog’s enhancements are not merely additive; they originate from the synergistic combination of its different modules.

Uncharted territory: groundbreaking study

A unified framework for detecting point and collective anomalies in logs optimizing the process and guaranteeing consistency in the analysis. This comprehensive method reduces blind spots that may arise from examining each anomaly type in isolation and improves the accuracy and reliability of the detection system. Utilizing a unified framework enables to deployment of resources more efficiently and addresses anomalies, hence enhancing the overall resilience. The unified framework enables the application of different modalities for improved and informative point and collective anomaly detection. As the discussion in Section “Classification”, CoLog assigns labels to log messages and the background or context of log events. Subsequently, upon receiving input *I*, it aims to learn and predict the label associated with each input. This method will involve four classes. Where 0 indicates both modalities are negative, 1 signifies that only the log event is anomalous, 2 denotes that only the background or context of the log event is anomalous, and 3 represents a normal instance of input. The experimental results of groundbreaking study on the Casper, Jhuisi, Nssal, Honey7, Zookeeper, Hadoop, and BlueGene/L datasets are shown in Table 18. It can be seen that CoLog performs excellent performance on this task. CoLog achieves excellent stable outcomes across all metrics on all datasets. For example, CoLog achieved a recall of 99.94% on the zookeeper dataset. The efficacy of CoLog in detecting point and collective anomalies in log files is primarily related to its ability to learn representations from various modalities of log files via distinct transformer blocks along with its ability to capture interactions across these modalities. This approach enables the CoLog to utilize the informative characteristics of log modalities, including semantic and sequential features of logs. Through a collaboration tunnel, MHIA, MAL, and the balancing layer, CoLog can more efficiently detect both point and collective anomalies. This collaboration guarantees that no essential insights are ignored. Note that the development of CoLog for more modalities is straightforward.

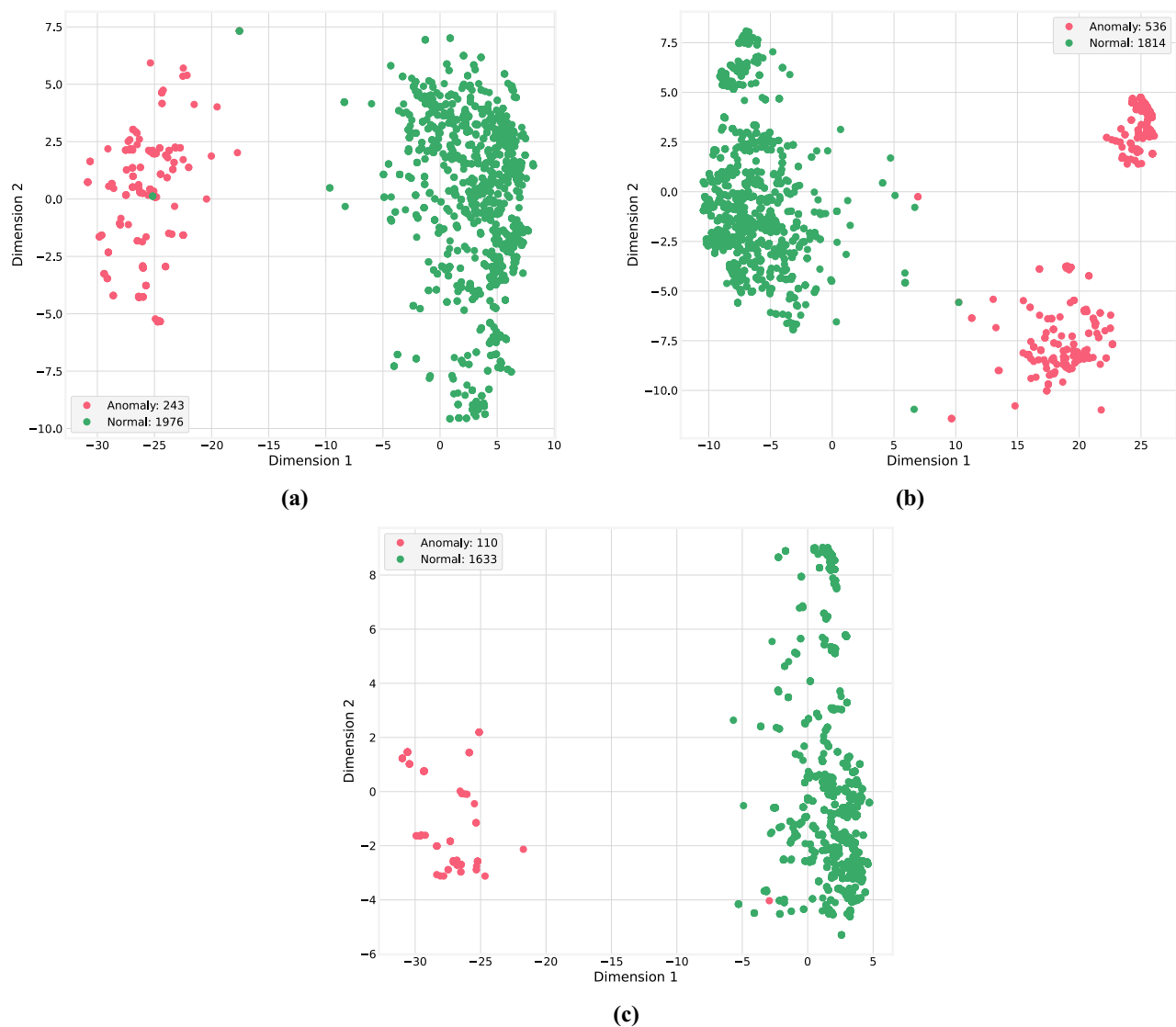


Fig. 18. Visual illustrations of the CoLog’s output vectors on the (a) Casper, (b) Jhuisi, and (c) Honey7 datasets utilizing PCA.

Dataset	Average	Class	# Samples	Precision	Recall	F1-Score	Accuracy
Honey5		0: Anomaly	56588	98.375	99.731	99.049	99.129
		1: Normal	67798	99.773	98.625	99.196	
	Macro		124386	99.074	99.178	99.122	
Windows		0: Anomaly	6404	99.855	96.705	98.255	99.119
		1: Normal	18567	98.876	99.952	99.411	
	Macro		24971	99.365	98.328	98.833	

Table 16. The evaluation of CoLog on previously unobserved datasets.

Contribution chronicles: comparative study of contributions

The comprehensive evaluation of contributions in existing literature compared to our study’s advancements underscores significant distinctions and synergies. FastLogAD⁷⁷ presents efficient utilization of normal data coupled with an innovative anomaly generation method. In contrast, our approach, CoLog, augments this by encoding log records across multiple modalities. The pylogsentiment³⁸ focuses on SA for log anomaly detection and addressing class imbalance. CoLog surpasses this by applying MSA to log anomaly detection that outperforms state-of-the-art methods. RAGLog⁹², a retrieval-augmented generation model, primarily

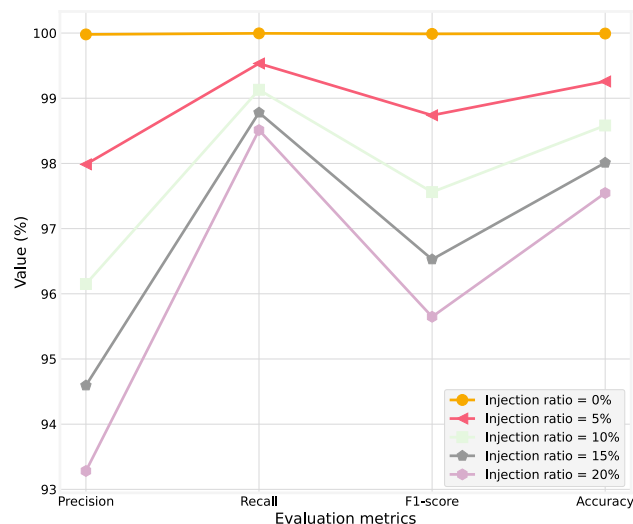


Fig. 19. The visual illustration of CoLog's outcomes for various injection ratios of unstable log events on the Spark dataset.

emphasizes storing normal log entries in a vector database. Our method, however, extends beyond mere storage to learn about signatures of abnormalities. UMFLog⁸⁴ employs a dual-model architecture integrating BERT for semantic feature extraction and VAE for statistical feature analysis, along with handling long log data sequences. CoLog similarly learns long sequences of log records effectively but transcends UMFLog by incorporating a collaborative approach to capture interactions between modalities and utilizing MAL to address heterogeneity. LogMS⁵⁹ implements a two-step model. A multi-source information fusion-based LSTM for anomaly detection followed by a GRU network for probability label estimation. Conversely, CoLog's unified framework for detecting abnormalities offers a streamlined and more comprehensive solution. MDFULog⁴⁵ addresses noise in log data and adopts an informer-based anomaly detection approach, whereas CoLog's modules, i.e., collaborative transformer and MAL, inherently mitigate noise. Finally, multi-feature fusion (MFF)⁴⁴ leverages late fusion for MFF-based anomaly detection by evaluating HTTP textual content, status code, and frequency features. Yet, CoLog's intermediate fusion mechanism significantly enhances performance metrics. Collectively, the contributions of CoLog not only embody advancements found in existing methods but also amalgamate their strengths within a cohesive, unified framework. Thus, CoLog's approach is not only multifaceted, addressing a wide spectrum of challenges presented by log anomaly detection, but also showcases superior performance metrics compared to individual contributions of FastLogAD, pylogsentiment, RAGLog, UMFLog, LogMS, MDFULog, and MFF. The comparative analysis in Table 19 distinctly illustrates how CoLog stands at the forefront of innovation. CoLog's ability to effectively encode log records utilizing various modalities, addressing heterogeneity through MAL, and demonstrating exceptional performance in detecting both point and collective abnormalities within a unified architecture validates CoLog as an innovative approach in the field of log anomaly detection.

Conclusion

In this paper, we introduce CoLog, a novel framework for collaborative encoding and anomaly detection that utilizes various log modalities. CoLog employs a supervised learning methodology, with its outcomes in log anomaly detection highlighting its scholarly significance in establishing an upper performance limit for this domain. With adequate labeled data, CoLog develops a standard of "theoretical accuracy" for evaluating unsupervised or semi-supervised approaches. Consequently, CoLog serves as a reference baseline for forthcoming research trajectories, offering comparative value and direction for the development of more efficient models. Additionally, the proposed approach addresses the inherent heterogeneity of log data by implementing MAL, ensuring robust performance across multiple benchmarks. CoLog's ability to detect both point and collective anomalies within a unified framework distinguishes it from conventional methods that focus solely on one type of anomaly. This comprehensive detection capability underpins the versatility and adaptability of CoLog, making it a significant advancement in the field of log anomaly detection. The potential applications of CoLog are vast, spanning from enhancing security measures through more accurate anomaly detection in cybersecurity logs to improving operational efficiency by identifying system irregularities in real-time. Additionally, its robust performance under varying noise conditions demonstrates its reliability and effectiveness in diverse environments.

Limitations

Real-time application: The present study assesses CoLog in a batch-processing mode. Real-time anomaly detection frequently entails supplementary constraints, like latency and limitations on computational resources. Examining and enhancing CoLog's performance in real-time settings is essential for effective implementation.

Dataset	Modality		Module			Metric			
	Seq	Sem	MHIA	MAL	BL	Precision	Recall	F1-score	Accuracy
Casper	✗	✓	✗	✗	✗	98.80	99.85	99.31	99.73
	✓	✗	✗	✗	✗	98.41	99.10	98.74	98.84
	✓	✓	✓	✗	✓	99.39	99.92	99.65	99.86
	✓	✓	✓	✗	✗	99.33	99.51	99.42	99.77
	✓	✓	✓	✓	✗	99.33	99.51	99.42	99.77
	✓	✓	✗	✓	✗	99.97	99.79	99.88	99.95
	✓	✓	✗	✓	✓	99.36	99.72	99.54	99.82
	✓	✓	✗	✗	✓	99.56	99.74	99.65	99.86
	✓	✓	✓	✓	✓	100	100	100	100
Jhuisi	✗	✓	✗	✗	✗	99.88	99.88	99.88	99.92
	✓	✗	✗	✗	✗	96.88	92.39	94.51	99.12
	✓	✓	✓	✗	✓	99.61	99.54	99.58	99.70
	✓	✓	✓	✗	✗	97.98	98.29	98.13	98.68
	✓	✓	✓	✓	✗	99.22	99.09	99.15	99.40
	✓	✓	✗	✓	✗	99.42	99.49	99.46	99.62
	✓	✓	✗	✓	✓	99.77	99.51	99.64	99.74
	✓	✓	✗	✗	✓	99.63	99.04	99.33	99.53
	✓	✓	✓	✓	✓	100	100	100	100
Honey7	✗	✓	✗	✗	✗	99.97	99.55	99.76	99.94
	✓	✗	✗	✗	✗	96.92	99.05	97.93	98.52
	✓	✓	✓	✗	✓	99.97	99.55	99.76	99.94
	✓	✓	✓	✗	✗	99.94	99.09	99.51	99.89
	✓	✓	✓	✓	✗	99.97	99.55	99.76	99.94
	✓	✓	✗	✓	✗	100	100	100	100
	✓	✓	✗	✓	✓	99.97	99.55	99.76	99.94
	✓	✓	✗	✗	✓	99.51	99.51	99.51	99.89
	✓	✓	✓	✓	✓	100	100	100	100

Table 17. Ablation study of CoLog. Seq and Sem denote sequence and semantic, respectively. Also, CT, MHIA, MAL, and BL denote collaborative transformer, multi-head impressed attention, modality adaptation layer, and balancing layer.

Dataset	Precision	Recall	F1-score	Accuracy
Casper	99.43	99.40	99.41	99.64
Jhuisi	99.82	99.48	99.65	99.70
Nssal	99.32	99.70	99.51	99.91
Honey7	99.77	98.90	99.33	99.77
Zookeeper	99.98	99.94	99.96	99.99
Hadoop	99.17	99.84	99.50	99.98

Table 18. The experimental results of a groundbreaking study on all training datasets.

Log entries that are not understandable by humans: Certain log entries, especially those from operating systems, may be difficult for humans to decipher, presenting a difficulty for first-time interpretation and labeling of such data.

Adapting to evolving log structures: The logs develop over time with system updates, necessitating continual adaptation and retraining of the model to preserve validity.

The Capacity for Noise Resilience: While CoLog exhibits robustness to varying noise levels, certain extreme scenarios or certain noise types may still substantially impair the performance.

Paper	Year	Methodology	Datasets Used	Mean F1-score	Key Contributions
FastLogAD ⁷⁷	2024	Sequence-based unimodal unsupervised learning	1) HDFS ^{1 12} 2) BlueGene/L 3) Thunderbird ¹³²	94.18	1) Efficient utilization of normal data. 2) Innovative anomaly generation method.
pylogsentiment ³⁸	2020	Semantic-based unimodal supervised learning	1) Spark 2) Honey5 3) Windows 4) Casper 5) Jhuisi 6) Nssal 7) Honey7 8) Zookeeper 9) Hadoop 10) BlueGene/L	99.14	1) Implements sentiment analysis for log anomaly detection. 2) Addressing class imbalance.
RAGLog ⁹²	2024	LLM-based unimodal unsupervised learning	1) BlueGene/L 2) Thunderbird	89.00	1) A retrieval-augmented generation model tdat employs a vector database to store normal log entries.
UMFLog ⁸⁴	2023	Independent network-based ultimodal unsupervised learning	1) HDFS 2) BlueGene/L 3) Thunderbird	99.56	1) Employs a dual-model architecture with BERT for semantic feature extraction and VAE for statistical feature analysis. 2) Handles long sequences of log data.
LogMS ⁵⁹	2024	Early fusion-based multimodal unsupervised & semi-supervised learning	1) HDFS 2) BlueGene/L	99.10	1) Employs a two-step model. The first step uses a multi-source information fusion-based LSTM to detect anomalies by utilizing semantic, sequential, and quantitative data. Following that, a probability label estimation-based GRU network is used.
MDFULog ⁴⁵	2023	Intermediate fusion-based multimodal supervised learning	1) HDFS 2) OpenStack ¹³²	97.00	1) Addresses noise in log data. 2) Informer-based anomaly detection.
MFF ⁴⁴	2023	Late fusion-based multimodal supervised learning	1) LLSD2 ⁴⁴	93.10	1) Detects web scanning behavior by considering HTTP textual content, status code, and frequency features. 2) Employ late fusion MFF-based network to detect anomalies.
CoLog	2025	Intermediate fusion-based multimodal supervised learning	1) Spark 2) Honey5 3) Windows 4) Casper 5) Jhuisi 6) Nssal 7) Honey7 8) Zookeeper 9) Hadoop 10) BlueGene/L	99.99	1) Encodes log records collaboratively according to various log modalities. 2) Employs MAL to address heterogeneity among modalities. 3) Outperforms state-of-the-art methods. 4) Detects point and collective abnormalities within a unified framework.

Table 19. The comprehensive overview of contributions in existing literature compared to our study. ¹Hadoop Distributed File System.

Future works

Future work could explore the integration of CoLog with real-time monitoring systems to enable proactive anomaly management and automated responses to detected anomalies on industrial systems. Moreover, extending CoLog’s application to other domains, such as financial fraud detection or health monitoring, could further validate its efficacy and adaptability. Since CoLog assumes a moderately imbalanced dataset and stable log templates, which generally hold in benchmark datasets but may vary in dynamic production systems. In extreme imbalance scenarios (e.g., anomaly ratio<1%) or in environments with rapidly evolving templates, performance may degrade due to representation drift. To mitigate this, CoLog’s balancing layer can be combined with adaptive resampling, online calibration, or few-shot fine-tuning. We also note that continual learning extensions could further improve adaptability. Future work will explore these directions to ensure robustness under diverse operational conditions. Additionally, alternative modalities of log data, including quantitative information, may also be employed in the anomaly detection process. The development of CoLog for additional modalities is straightforward. Also, investigating adaptive mechanisms for better managing various types of noise could be a beneficial avenue. The promising performance and broad applicability of CoLog suggest that it holds considerable potential for driving advancements in multiple industries where accurate and timely anomaly detection is critical. By addressing current limitations in log analysis, CoLog paves the way for more sophisticated and comprehensive solutions to complex data challenges.

Data availability

The datasets generated and/or analysed during the current study are available in the loghub repository at <https://github.com/logpai/loghub> and in the pylogsentiment repository at <https://github.com/studiawan/pylogsentiment>.

Materials availability

The datasets, computing resources, algorithms, and libraries employed in this AI development can be obtained upon request, contingent upon relevant access policies and license agreements.

Code availability

The paper’s implementation code is openly accessible on GitHub at <https://github.com/NasirzadehMoh/CoLog>, guaranteeing transparency and reproducibility for ongoing development and collaboration.

Received: 10 May 2025; Accepted: 5 November 2025
Published online: 28 December 2025

References

- Samariya, D. & Thakkar, A. A comprehensive survey of anomaly detection algorithms. *Annals of Data Science* **10**(3), 829–850. <https://doi.org/10.1007/s40745-021-00362-9> (2023).
- Zhao, X., Rodrigues, K., Luo, Y., Yuan, D. & Stumm, M. Non-Intrusive performance profiling for entire software stacks based on the flow reconstruction principle. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, 603–618. (USENIX Association, Savannah, GA, 2016). <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/zhao>
- Yu, X. et al. Cloudseer: Workflow monitoring of cloud infrastructures via interleaved logs. *SIGARCH Comput. Archit. News* **44**(2), 489–502. <https://doi.org/10.1145/2980024.2872407> (2016).
- Lin, Q., Zhang, H., Lou, J.-G., Zhang, Y. & Chen, X. Log clustering based problem identification for online service systems. In *Proceedings of the 38th International Conference on Software Engineering Companion. ICSE '16*, 102–111. (Association for Computing Machinery, New York, NY, USA, 2016). <https://doi.org/10.1145/2889160.2889232>.
- Roy, S., K nig, A.C., Dvorkin, I. & Kumar, M. Perfaugur: Robust diagnostics for performance anomalies in cloud services. In *2015 IEEE 31st International Conference on Data Engineering*, 1167–1178 (2015). <https://doi.org/10.1109/ICDE.2015.7113365>
- Oprea, A., Li, Z., Yen, T.-F., Chin, S.H. & Alrwais, S. Detection of early-stage enterprise infection by mining large-scale log data. In *2015 45th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*, 45–56 (2015). <https://doi.org/10.1109/DSN.2015.14>
- Beschastnikh, I., Brun, Y., Ernst, M.D., & Krishnamurthy, A. Inferring models of concurrent systems from logs of their behavior with csight. In *Proceedings of the 36th International Conference on Software Engineering. ICSE 2014*, 468–479. (Association for Computing Machinery, New York, NY, USA, 2014). <https://doi.org/10.1145/2568225.2568246>.
- Yen, T.-F., et al. Beehive: large-scale log analysis for detecting suspicious activity in enterprise networks. In *Proceedings of the 29th Annual Computer Security Applications Conference. ACSAC '13*, 199–208. (Association for Computing Machinery, New York, NY, USA, 2013). <https://doi.org/10.1145/2523649.2523670>.
- Cinque, M., Cotroneo, D. & Pecchia, A. Event logs for the analysis of software failures: A rule-based approach. *IEEE Transactions on Software Engineering* **39**(6), 806–821. <https://doi.org/10.1109/TSE.2012.67> (2013).
- Lou, J.-G., Fu, Q., Yang, S., Xu, Y. & Li, J. Mining invariants from console logs for system problem detection. In: 2010 USENIX Annual Technical Conference (USENIX ATC 10) (2010)
- Lou, J.-G., Fu, Q., Yang, S., Li, J. & Wu, B. Mining program workflow from interleaved traces. In *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. KDD '10*, 613–622. (Association for Computing Machinery, New York, NY, USA, 2010). <https://doi.org/10.1145/1835804.1835883>.
- Xu, W., Huang, L., Fox, A., Patterson, D. & Jordan, M.I. Detecting large-scale system problems by mining console logs. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles. SOSP '09*, 117–132. (Association for Computing Machinery, New York, NY, USA, 2009). <https://doi.org/10.1145/1629575.1629587>.
- Xu, W., Huang, L., Fox, A., Patterson, D. & Jordan, M. Online system problem detection by mining patterns of console logs. In *2009 Ninth IEEE International Conference on Data Mining*, 588–597 (2009). <https://doi.org/10.1109/ICDM.2009.19>
- Fu, Q., Lou, J.-G., Wang, Y. & Li, J. Execution anomaly detection in distributed systems through unstructured log analysis. In *2009 Ninth IEEE International Conference on Data Mining*, 149–158 (2009). <https://doi.org/10.1109/ICDM.2009.60>
- Yamanishi, K., & Maruyama, Y. Dynamic syslog mining for network failure monitoring. In *Proceedings of the Eleventh ACM SIGKDD International Conference on Knowledge Discovery in Data Mining. KDD '05*, pp. 499–508. (Association for Computing Machinery, New York, NY, USA, 2005). <https://doi.org/10.1145/1081870.1081927>.
- Rouillard, J.P. Real-time log file analysis using the simple event correlator (sec). In: LISA, 4, 133–150 (2004)
- Prewett, J.E. Analyzing cluster log files using logsurfer. In *Proceedings of the 4th Annual Conference on Linux Clusters*, 1–12 (Citeseer State College, PA, USA, 2003)
- Hansen, S.E. & Atkins, E.T. Automated system monitoring and notification with swatch. In *LISA*, **93**, 145–152 (Monterey, CA, 1993).
- Memon, A.U., Cordy, J.R., & Dean, T. Log File Categorization and Anomaly Analysis Using Grammar Inference
- Cui, T., et al. LogEval: A Comprehensive Benchmark Suite for Large Language Models In Log Analysis (2024). arxiv.org/abs/2407.01896
- Du, Q., Zhao, L., Xu, J., Han, Y. & Zhang, S. Log-based anomaly detection with multi-head scaled dot-product attention mechanism. In (ed. Strauss, C., Kotsis, G., Tjoa, A.M. & Khalil, I.) *Database and Expert Systems Applications*, 335–347. (Springer, Cham, 2021). https://doi.org/10.1007/978-3-030-86472-9_31
- Le, V.-H. & Zhang, H. Log-based anomaly detection with deep learning: how far are we? In *Proceedings of the 44th International Conference on Software Engineering. ICSE '22*, 1356–1367. (Association for Computing Machinery, New York, NY, USA, 2022). <https://doi.org/10.1145/3510003.3510155>.
- Pithode, K. & Pathoja, P.S. A study on log anomaly detection using deep learning techniques. In *2022 International Conference on Applied Artificial Intelligence and Computing (ICAIAIC)*, 290–298 (2022). <https://doi.org/10.1109/ICAIAIC53929.2022.9793238>
- Chen, Z., Liu, J., Gu, W., Su, Y. & Lyu, M.R. Experience Report: Deep Learning-based System Log Analysis for Anomaly Detection (2022). arxiv.org/abs/2107.05908
- Cheng, X., et al. Lift yourself up: Retrieval-augmented text generation with self-memory. In (ed. Oh, A., et al.) *Advances in Neural Information Processing Systems*, **36**, 43780–43799. Curran Associates, Inc., ??? (2023)
- Huang, S.-C. et al. Self-supervised learning for medical image classification: a systematic review and implementation guidelines. *NPJ Digital Medicine* **6**(1), 74. <https://doi.org/10.1038/s41746-023-00811-0> (2023).
- Jeong, J., et al. Multimodal image-text matching improves retrieval-based chest x-ray report generation. In (ed. Oguz, I., et al.) *Medical Imaging with Deep Learning. Proceedings of Machine Learning Research*, **227**, 978–990. PMLR, ??? (2024). <https://proceedings.mlr.press/v227/jeong24a.html>
- Le, M., et al. Voicebox: Text-guided multilingual universal speech generation at scale. In (ed. Oh, A., et al.) *Advances in Neural Information Processing Systems*, **36**, 14005–14034. Curran Associates, Inc., ??? (2023)
- Liu, Z., et al. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 10012–10022 (2021). [arXiv:2103.14030](https://arxiv.org/abs/2103.14030)
- Liu, Z., et al. Video swin transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 3202–3211 (2022). [arXiv:2106.13230](https://arxiv.org/abs/2106.13230)
- Saleki, S. & Tahmoresnezhad, J. Agry: a comprehensive framework for plant diseases classification via pretrained efficientnet and convolutional neural networks for precision agriculture. *Multimedia Tools and Applications* **83**(24), 64813–64851 (2024).
- Tahmoresnezhad, J. & Hashemi, S. Visual domain adaptation via transfer feature learning. *Knowledge and information systems* **50**(2), 585–605 (2017).
- Khorshidpour, Z., Tahmoresnezhad, J., Hashemi, S. & Hamzeh, A. Domain invariant feature extraction against evasion attack. *International Journal of Machine Learning and Cybernetics* **9**(12), 2093–2104 (2018).
- Khodadadi, M. & Tahmoresnezhad, J. HyMo: Vulnerability Detection in Smart Contracts using a Novel Multi-Modal Hybrid Model (2023). arxiv.org/abs/2304.13103
- Almodovar, C., Sabrina, F., Karimi, S. & Azad, S. Logfit: Log anomaly detection using fine-tuned language models. *IEEE Transactions on Network and Service Management* **21**(2), 1715–1723. <https://doi.org/10.1109/TNSM.2024.3358730> (2024).

36. Du, M., Li, F., Zheng, G. & Srikumar, V. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. CCS '17*, 1285–1298. (Association for Computing Machinery, New York, NY, USA, 2017). <https://doi.org/10.1145/3133956.3134015>.
37. Guo, H. et al. Logformer: A pre-train and tuning pipeline for log anomaly detection. *Proceedings of the AAAI Conference on Artificial Intelligence* **38**(1), 135–143. <https://doi.org/10.1609/aaai.v38i1.27764> (2024).
38. Studiawan, H., Sohel, F. & Payne, C. Anomaly detection in operating system logs with deep learning-based sentiment analysis. *IEEE Transactions on Dependable and Secure Computing* **18**(5), 2136–2148. <https://doi.org/10.1109/TDSC.2020.3037903> (2021).
39. Wang, S., Jiang, R., Wang, Z. & Zhou, Y. Deep Learning-based Anomaly Detection and Log Analysis for Computer Networks (2024). arxiv.org/abs/2407.05639
40. Landauer, M., Onder, S., Skopik, F. & Wurzenberger, M. Deep learning for anomaly detection in log data: A survey. *Machine Learning with Applications* **12**, 100470. <https://doi.org/10.1016/j.mlwa.2023.100470> (2023).
41. Li, H. & Li, Y. Logspy: System log anomaly detection for distributed systems. In *2020 International Conference on Artificial Intelligence and Computer Engineering (ICAICE)*, 347–352 (2020). <https://doi.org/10.1109/ICAICE51518.2020.00073>
42. Wang, Q., Zhang, X., Wang, X. & Cao, Z. Log sequence anomaly detection method based on contrastive adversarial training and dual feature extraction. *Entropy* **24**(1) (2022) <https://doi.org/10.3390/e24010069>
43. Qi, J. et al. Logencoder: Log-based contrastive representation learning for anomaly detection. *IEEE Transactions on Network and Service Management* **20**(2), 1378–1391. <https://doi.org/10.1109/TNSM.2023.3239522> (2023).
44. Zhao, Q., et al. A multi-feature fusion method for web scanning behavior detection in online web logs. In *2023 4th International Conference on Computers and Artificial Intelligence Technology (CAIT)*, 246–252 (2023). <https://doi.org/10.1109/CAIT59945.2023.10469208>
45. Li, M., Sun, M., Li, G., Han, D. & Zhou, M. Mdfulog: Multi-feature deep fusion of unstable log anomaly detection model. *Applied Sciences* **13**(4) (2023) <https://doi.org/10.3390/app13042237>
46. Lu, S., Wei, X., Li, Y. & Wang, L. Detecting anomaly in big data system logs using convolutional neural network. In *2018 IEEE 16th Intl Conf on Dependable, Autonomic and Secure Computing, 16th Intl Conf on Pervasive Intelligence and Computing, 4th Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress (DASC/PiCom/DataCom/CyberSciTech)*, 151–158 (2018). <https://doi.org/10.1109/DASC/PiCom/DataCom/CyberSciTech.2018.00037>
47. Wang, Z., Tian, J., Fang, H., Chen, L. & Qin, J. Lightlog: A lightweight temporal convolutional network for log anomaly detection on the edge. *Computer Networks* **203**, 108616. <https://doi.org/10.1016/j.comnet.2021.108616> (2022).
48. Zhang, L., et al. Logattn: Unsupervised log anomaly detection with an autoencoder based attention mechanism. In (ed. Qiu, H., Zhang, C., Fei, Z., Qiu, M. & Kung, S.-Y.) *Knowledge Science, Engineering and Management*, 222–235. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-82153-1_19
49. Hashemi, S. & M ntyl, M. Onelog: towards end-to-end software log anomaly detection. *Automated Software Engineering* **31**(2) (2024) <https://doi.org/10.1007/s10515-024-00428-x>
50. Xiao, C., Huang, J. & Wu, W. Detecting anomalies in cluster system using hybrid deep learning model. In (ed. Shen, H. & Sang, Y.) *Parallel Architectures, Algorithms and Programming*, 393–404. Springer, Singapore (2020). https://doi.org/10.1007/978-981-15-2767-8_35
51. Wang, P., Zhang, X. & Cao, Z. Log anomaly detection based on semantic features and topic features. In (ed. Tari, Z., Li, K. & Wu, H.) *Algorithms and Architectures for Parallel Processing*, 407–427. (Springer, Singapore, 2024). https://doi.org/10.1007/978-981-97-0808-6_24
52. Farzad, A. Log message anomaly detection with oversampling. *International Journal of Artificial Intelligence and Applications (IJAIA)* **11**(4) (2020)
53. Farzad, A. & Gulliver, T.A. Log Message Anomaly Detection and Classification Using Auto-B/LSTM and Auto-GRU (2021). arxiv.org/abs/1911.08744
54. Gu, S., et al. Research on system log anomaly detection combining two-way slice gru and ga-attention mechanism. In *2021 4th International Conference on Artificial Intelligence and Big Data (ICAIBD)*, 577–583 (2021). <https://doi.org/10.1109/ICAIBD5199.0.2021.9459087>
55. Li, X., Chen, P., Jing, L., He, Z. & Yu, G. Swisslog: Robust and unified deep learning based log anomaly detection for diverse faults. In *2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*, 92–103 (2020). <https://doi.org/10.1109/ISSRE5003.2020.00018>
56. Wang, Z., et al. Multi-scale one-class recurrent neural networks for discrete event sequence anomaly detection. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining. KDD '21*, 3726–3734. (Association for Computing Machinery, New York, NY, USA, 2021). <https://doi.org/10.1145/3447548.3467125>.
57. Yang, L., et al. Semi-supervised log-based anomaly detection via probabilistic label estimation. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, 1448–1460 (2021). <https://doi.org/10.1109/ICSE43902.2021.00130>
58. Zhang, D., Dai, D., Han, R. & Zheng, M. Sentilog: Anomaly detecting on parallel file systems via log-based sentiment analysis. In *Proceedings of the 13th ACM Workshop on Hot Topics in Storage and File Systems. HotStorage '21*, 86–93. (Association for Computing Machinery, New York, NY, USA, 2021). <https://doi.org/10.1145/3465332.3470873>.
59. Yu, Z., et al. Logms: a multi-stage log anomaly detection method based on multi-source information fusion and probability label estimation. *Frontiers in Physics* **12** (2024) <https://doi.org/10.3389/fphy.2024.1401857>
60. Zhang, C. et al. Layerlog: Log sequence anomaly detection based on hierarchical semantics. *Applied Soft Computing* **132**, 109860. <https://doi.org/10.1016/j.asoc.2022.109860> (2023).
61. Fu, Y., Liang, K. & Xu, J. Mlog: Mogrifier lstm-based log anomaly detection approach using semantic representation. *IEEE Transactions on Services Computing* **16**(5), 3537–3549. <https://doi.org/10.1109/TSC.2023.3289488> (2023).
62. Yu, D., et al. Anomaly detection in unstructured logs using attention-based bi-lstm network. In *2021 7th IEEE International Conference on Network Intelligence and Digital Content (IC-NIDC)*, 403–407 (2021). <https://doi.org/10.1109/IC-NIDC54101.2021.9660476>
63. Zhang, X., et al. Robust log-based anomaly detection on unstable log data. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ESEC/FSE 2019*, 807–817. (Association for Computing Machinery, New York, NY, USA, 2019). <https://doi.org/10.1145/3338906.3338931>.
64. Yang, R., Qu, D., Gao, Y., Qian, Y. & Tang, Y. Nlsalog: An anomaly detection framework for log sequence in security management. *IEEE Access* **7**, 181152–181164. <https://doi.org/10.1109/ACCESS.2019.2953981> (2019).
65. Niu, W. et al. A robust wide & deep learning framework for log-based anomaly detection. *Applied Soft Computing* **153**, 111314. <https://doi.org/10.1016/j.asoc.2024.111314> (2024).
66. Yang, H., et al. An anomaly detection algorithm for logs based on self-attention mechanism and bigru model. In: (ed. Jia, Y., Zhang, W., Fu, Y. & Wang, J.) *Proceedings of 2023 Chinese Intelligent Systems Conference*, 877–888. (Springer, Singapore, 2023). https://doi.org/10.1007/978-981-99-6847-3_76
67. Niu, W. et al. Fsmlog: Discovering anomalous logs combining full semantic information and multifeature fusion. *IEEE Internet of Things Journal* **11**(3), 4442–4453. <https://doi.org/10.1109/JIOT.2023.3300690> (2024).
68. Otomo, K., Kobayashi, S., Fukuda, K. & Esaki, H. Latent variable based anomaly detection in network system logs. *IEICE TRANSACTIONS on Information and Systems* **102**(9), 1644–1652. <https://doi.org/10.1587/transinf.2018OFP0007> (2019).
69. Qian, Y., Ying, S. & Wang, B. Anomaly detection in distributed systems via variational autoencoders. In *2020 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 2822–2829 (2020). <https://doi.org/10.1109/SMC42975.2020.9283078>

70. Wadekar, A., Gupta, T., Vijan, R. & Kazi, F. Hybrid cae-vae for unsupervised anomaly detection in log file systems. In *2019 10th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, 1–7 (2019). <https://doi.org/10.1109/ICCCNT45670.2019.8944863>
71. Wang, X. et al. Robust log anomaly detection based on contrastive learning and multi-scale mass. *The Journal of Supercomputing* **78**(16), 17491–17512. <https://doi.org/10.1007/s11227-022-04508-1> (2022).
72. Catillo, M., Pecchia, A. & Villano, U. Autolog: Anomaly detection by deep autoencoding of system logs. *Expert Systems with Applications* **191**, 116263. <https://doi.org/10.1016/j.eswa.2021.116263> (2022).
73. Han, X. & Yuan, S. Unsupervised cross-system log anomaly detection via domain adaptation. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management. CIKM '21*, 3068–3072. (Association for Computing Machinery, New York, NY, USA, 2021). <https://doi.org/10.1145/3459637.3482209>.
74. Xia, B., Bai, Y., Yin, J., Li, Y. & Xu, J. Loggan: a log-level generative adversarial network for anomaly detection using permutation event modeling. *Information Systems Frontiers* **23**, 285–298. <https://doi.org/10.1007/s10796-020-10026-3> (2021).
75. Zhao, Z. et al. Trine: Syslog anomaly detection with three transformer encoders in one generative adversarial network. *Applied Intelligence* **52**(8), 8810–8819. <https://doi.org/10.1007/s10489-021-02863-9> (2022).
76. Yamanaka, Y., Takahashi, T., Minami, T. & Nakajima, Y. LogELECTRA: Self-supervised Anomaly Detection for Unstructured Logs (2024). arxiv.org/abs/2402.10397
77. Lin, Y., Deng, H. & Li, X. FastLogAD: Log Anomaly Detection with Mask-Guided Pseudo Anomaly Generation and Discrimination (2024). arxiv.org/abs/2404.08750
78. Qi, J., et al. Adanomaly: Adaptive anomaly detection for system logs with adversarial learning. In *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*, 1–5 (2022). <https://doi.org/10.1109/NOMS54207.2022.9789917>
79. Guo, H., Yuan, S. & Wu, X. Logbert: Log anomaly detection via bert. In *2021 International Joint Conference on Neural Networks (IJCNN)*, 1–8 (2021). <https://doi.org/10.1109/IJCNN52387.2021.9534113>
80. Nedelkoski, S., Bogatinovski, J., Acker, A., Cardoso, J. & Kao, O. Self-attentive classification-based anomaly detection in unstructured logs. In *2020 IEEE International Conference on Data Mining (ICDM)*, 1196–1201 (2020). <https://doi.org/10.1109/ICDM50108.2020.00148>
81. Zang, R., et al. MLAD: A Unified Model for Multi-system Log Anomaly Detection (2024). arxiv.org/abs/2401.07655
82. Wittkopp, T., et al. A2Log: Attentive Augmented Log Anomaly Detection (2021). arxiv.org/abs/2109.09537
83. Tan, X., Han, N., Lu, S., Chen, W. & Wang, D. Semlog: A semantics-based approach for anomaly detection in big data system logs. In *2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS)*, 1199–1206 (2023). <https://doi.org/10.1109/ICPADS60453.2023.00174>
84. He, S., Deng, T., Chen, B., Sherratt, R.S. & Wang, J. Unsupervised log anomaly detection method based on multi-feature. *Computers, Materials & Continua* **76**(1) (2023) <https://doi.org/10.32604/cmc.2023.037392>
85. Han, C. et al. Few-shot log anomaly detection based on matching networks. *IEEE Transactions on Network and Service Management* **21**(3), 2909–2925. <https://doi.org/10.1109/TNSM.2024.3363626> (2024).
86. Zhang, L., et al. Multivariate log-based anomaly detection for distributed database. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. KDD '24*, 4256–4267. (Association for Computing Machinery, New York, NY, USA, 2024). <https://doi.org/10.1145/3637528.3671725>.
87. Ede, T.v., et al. Deepcase: Semi-supervised contextual analysis of security events. In *2022 IEEE Symposium on Security and Privacy (SP)*, 522–539 (2022). <https://doi.org/10.1109/SP46214.2022.9833671>
88. Li, X., et al. Improving performance of log anomaly detection with semantic and time features based on bilstm-attention. In *2021 2nd International Conference on Electronics, Communications and Information Technology (CECIT)*, 661–666 (2021). <https://doi.org/10.1109/CECIT53797.2021.00121>
89. Guo, Y., Wen, Y., Jiang, C., Lian, Y. & Wan, Y. Detecting Log Anomalies with Multi-Head Attention (LAMA) (2021). arxiv.org/abs/2101.02392
90. Wan, Y., Liu, Y., Wang, D. & Wen, Y. Glad-paw: Graph-based log anomaly detection by position aware weighted graph attention network. In (ed. Karlapalem, K. et al.) *Advances in Knowledge Discovery and Data Mining*, 66–77. (Springer, Cham, 2021). https://doi.org/10.1007/978-3-030-75762-5_6
91. Decker, L., Leite, D., Viola, F. & Bonacorsi, D. Comparison of evolving granular classifiers applied to anomaly detection for predictive maintenance in computing centers. In *2020 IEEE Conference on Evolving and Adaptive Intelligent Systems (EAIS)*, 1–8 (2020). <https://doi.org/10.1109/EAIS48028.2020.9122779>
92. Pan, J., Liang, W.S. & Yidi, Y. Raglog: Log anomaly detection using retrieval augmented generation. In *2024 IEEE World Forum on Public Safety Technology (WFPST)*, 169–174 (2024). <https://doi.org/10.1109/WFPST58552.2024.00034>
93. Liu, Y., et al. Interpretable online log analysis using large language models with prompt strategies. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension. ICPC '24*, 35–46. (Association for Computing Machinery, New York, NY, USA, 2024). <https://doi.org/10.1145/3643916.3644408>.
94. Liu, Z., Zhou, B., Chu, D., Sun, Y. & Meng, L. Modality translation-based multimodal sentiment analysis under uncertain missing modalities. *Information Fusion* **101**, 101973. <https://doi.org/10.1016/j.inffus.2023.101973> (2024).
95. Deng, Y., Li, Y., Xian, S., Li, L. & Qiu, H. Mual: Enhancing multimodal sentiment analysis with cross-modal attention and difference loss. *International Journal of Multimedia Information Retrieval* **13**(3), 31. <https://doi.org/10.1007/s13735-024-00340-w> (2024).
96. Ahuja, G., Alaei, A. & Pal, U. A new multimodal sentiment analysis for images containing textual information. *Multimedia Tools and Applications*, 1–30 (2024) <https://doi.org/10.1007/s11042-024-19999-8>
97. Wang, D. et al. Tetfn: A text enhanced transformer fusion network for multimodal sentiment analysis. *Pattern Recognition* **136**, 109259. <https://doi.org/10.1016/j.patcog.2022.109259> (2023).
98. Li, Z. et al. Multimodal sentiment analysis based on interactive transformer and soft mapping. *Wireless Communications and Mobile Computing* **2022**(1), 6243347. <https://doi.org/10.1155/2022/6243347> (2022).
99. Hu, G., et al. UniMSE: Towards Unified Multimodal Sentiment Analysis and Emotion Recognition (2022). arxiv.org/abs/2211.11256
100. Wang, Z., Wan, Z. & Wan, X. Transmodality: An end2end fusion method with transformer for multimodal sentiment analysis. In *Proceedings of The Web Conference 2020. WWW '20*, 2514–2520. (Association for Computing Machinery, New York, NY, USA, 2020). <https://doi.org/10.1145/3366423.3380000>.
101. Delbrouck, J.-B., Tits, N., Brousmiche, M. & Dupont, S. A transformer-based joint-encoding for emotion recognition and sentiment analysis. In: *Second Grand-Challenge and Workshop on Multimodal Language (Challenge-HML)*. Association for Computational Linguistics, ??? (2020). <https://doi.org/10.18653/v1/2020.challengehml-1.1>.
102. Zadeh, A., Chen, M., Poria, S., Cambria, E. & Morency, L.-P. Tensor Fusion Network for Multimodal Sentiment Analysis (2017). arxiv.org/abs/1707.07250
103. Chen, M., et al. Multimodal sentiment analysis with word-level fusion and reinforcement learning. In *Proceedings of the 19th ACM International Conference on Multimodal Interaction. ICMI '17*, 163–171. (Association for Computing Machinery, New York, NY, USA, 2017). <https://doi.org/10.1145/3136755.3136801>.
104. Yu, Z., Yu, J., Cui, Y., Tao, D. & Tian, Q. Deep modular co-attention networks for visual question answering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2019)

105. Li, Z., Xiang, Z., Gong, W. & Wang, H. Unified model for collective and point anomaly detection using stacked temporal convolution networks. *Applied Intelligence* **52**(3), 3118–3131. <https://doi.org/10.1007/s10489-021-02559-0> (2022).
106. Liu, H., Zhang, H.-S., Tang, Y. & Yao, Y. A unified detection approach for point and subsequence anomaly data from train axle temperature sensors. *IEEE Sensors Journal* **23**(20), 24772–24786. <https://doi.org/10.1109/JSEN.2023.3307623> (2023).
107. Vaswani, A., et al. Attention is all you need. *Advances in neural information processing systems* **30** (2017).
108. Chalapathy, R., & Chawla, S. Deep Learning for Anomaly Detection: A Survey (2019). arxiv.org/abs/1901.03407
109. Liang, Y., Zhang, Y., Xiong, H., & Sahoo, R. Failure prediction in ibm bluegene/l event logs. In *Seventh IEEE International Conference on Data Mining (ICDM 2007)*, 583–588 (2007). <https://doi.org/10.1109/ICDM.2007.46>
110. Chen, M., Zheng, A.X., Lloyd, J., Jordan, M.I. & Brewer, E. Failure diagnosis using decision trees. In *International Conference on Autonomic Computing, 2004. Proceedings.*, 36–43 (2004). <https://doi.org/10.1109/ICAC.2004.1301345>
111. He, S., Zhu, J., He, P. & Lyu, M.R. Experience report: System log analysis for anomaly detection. In *2016 IEEE 27th International Symposium on Software Reliability Engineering (ISSRE)*, 207–218 (2016). <https://doi.org/10.1109/ISSRE.2016.21>
112. Reimers, N. & Gurevych, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks (2019). arxiv.org/abs/1908.10084
113. Zhang, L., et al. E-log: Fine-grained elastic log-based anomaly detection and diagnosis for databases. *IEEE Transactions on Services Computing* (2025)
114. Qiu, K., Zhang, Y., Feng, Y. & Chen, F. Loganomex: An unsupervised log anomaly detection method based on electra-dp and gated bilinear neural networks. *Journal of Network and Systems Management* **33**(2), 1–29 (2025).
115. Qiu, K., Yan, M., Luo, T. & Chen, F. Fedaware: a distributed iot intrusion detection method based on fractal shrinking autoencoder. *Journal of King Saud University Computer and Information Sciences* **37**(7), 1–21 (2025).
116. Kingma, D.P. & Ba, J. Adam: A Method for Stochastic Optimization (2017). arxiv.org/abs/1412.6980
117. Minaee, S., et al. Large Language Models: A Survey (2024). arxiv.org/abs/2402.06196
118. Al-Zoghby, A. M., Al-Awadly, E. M. K., Ebada, A. I. & Awad, W. A. Overview of multimodal machine learning. *ACM Transactions on Asian and Low-Resource Language Information Processing* **24**(1), 1–20 (2025).
119. Liang, P. P., Zadeh, A. & Morency, L.-P. Foundations & trends in multimodal machine learning: Principles, challenges, and open questions. *ACM Computing Surveys* **56**(10), 1–42 (2024).
120. Qiu, K. et al. A multimodal sentiment analysis approach based on a joint chained interactive attention mechanism. *Electronics* **13**(10), 1922 (2024).
121. Su, Y., Tan, Y., An, S., Xing, M. & Feng, Z. Semantic-driven dual consistency learning for weakly supervised video anomaly detection. *Pattern Recognition* **157**, 110898. <https://doi.org/10.1016/j.patcog.2024.110898> (2025).
122. Su, Y., Li, J., An, S., Xing, M. & Feng, Z. Federated weakly-supervised video anomaly detection with mixture of local-to-global experts. *Information Fusion* **123**, 103256. <https://doi.org/10.1016/j.inffus.2025.103256> (2025).
123. Studiawan, H., Sohel, F. & Payne, C. Automatic log parser to support forensic analysis. In: *16th Australian Digital Forensics Conference* (2018)
124. He, P., Zhu, J., Zheng, Z. & Lyu, M.R. Drain: An online log parsing approach with fixed depth tree. In *2017 IEEE International Conference on Web Services (ICWS)*, 33–40 (2017). <https://doi.org/10.1109/ICWS.2017.13>
125. Mikolov, T., Chen, K., Corrado, G. & Dean, J. Efficient Estimation of Word Representations in Vector Space (2013). arxiv.org/abs/1301.3781
126. Nguyen, H. M., Cooper, E. W. & Kamei, K. Borderline over-sampling for imbalanced data classification. *International Journal of Knowledge Engineering and Soft Data Paradigms* **3**(1), 4–21 (2011).
127. He, H., Bai, Y., Garcia, E.A., & Li, S. Adasyn: Adaptive synthetic sampling approach for imbalanced learning. In *2008 IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence)*, 1322–1328 (2008). <https://doi.org/10.1109/IJCNN.2008.4633969>
128. Mani, I. & Zhang, I. knn approach to unbalanced data distributions: a case study involving information extraction. In *Proceedings of Workshop on Learning from Imbalanced Datasets*, **126**, 1–7, (ICML United States, 2003).
129. Ivan, T. An experiment with the edited nearest-neighbor rule. *IEEE Trans. Syst. Man Cybern.*; U.S.A.; DA. **6**(6), 448–452, (1976)
130. Seiffert, C., Khoshgoftaar, T. M., Van Hulse, J. & Napolitano, A. Rusboost: A hybrid approach to alleviating class imbalance. *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans* **40**(1), 185–197. <https://doi.org/10.1109/TSMCA.2009.2029559> (2010).
131. Hido, S., Kashima, H. & Takahashi, Y. Roughly balanced bagging for imbalanced data. *Statistical Analysis and Data Mining: The ASA Data Science Journal* **2**(5–6), 412–426. <https://doi.org/10.1002/sam.10061> (2009).
132. Zhu, J., He, S., He, P., Liu, J. & Lyu, M.R. Loghub: A large collection of system log datasets for ai-driven log analytics. In: *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, 355–366 (2023). <https://doi.org/10.1109/ISSRE5984.8.2023.00071>
133. Marty, R., Chuvakin, A., & Tricaud, S. Challenge 5 of the Honeynet project forensic challenge 2010-Log mysteries (2010). <https://www.honeynet.org/challenges/forensic-challenge-5-log-mysteries/>
134. Garfinkel, S. nps-2009-casper-rw: An ext3 file system from a bootable usb. Retrieved August 21, 2017 (2009)
135. Casey, E. & Richard III, G.G. Dfrws forensic challenge 2009. Retrieved August 21, 2017 (2009)
136. Arcas, G., Gonzales, H. & Cheng, J. Challenge 7 of the honeynet project forensic challenge 2011-forensic analysis of a compromised server. Retrieved August 21, 2017 (2011)
137. Lin, Q., Zhang, H., Lou, J.-G., Zhang, Y., & Chen, X. Log clustering based problem identification for online service systems. In *Proceedings of the 38th International Conference on Software Engineering Companion. ICSE '16*, 102–111. (Association for Computing Machinery, New York, NY, USA, 2016). <https://doi.org/10.1145/2889160.2889232>
138. Oliner, A. & Stearley, J. What supercomputers say: A study of five system logs. In *37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN'07)*, 575–584 (2007). <https://doi.org/10.1109/DSN.2007.103>
139. Paszke, A., et al.: An imperative style, high-performance deep learning library. In (ed. Wallach, H., Larochelle, H., Beygelzimer, A., Alché-Buc, F. & Fox, E., Garnett, R.) *Advances in Neural Information Processing Systems*, **32**. Curran Associates, Inc., ??? (2019)
140. Lemaître, G., Nogueira, F. & Aridas, C. K. Imbalanced-learn: A python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research* **18**(17), 1–5 (2017).
141. Liaw, R., et al. Tune: A Research Platform for Distributed Model Selection and Training (2018). arxiv.org/abs/1807.05118
142. Pedregosa, F., et al. Scikit-learn: Machine learning in python. *Journal of machine Learning research* **12**, 2825–2830 (2011)
143. Bodik, P., Goldszmidt, M., Fox, A., Woodard, D.B. & Andersen, H. Fingerprinting the datacenter: automated classification of performance crises. In *Proceedings of the 5th European Conference on Computer Systems. EuroSys '10*, 111–124, (Association for Computing Machinery, New York, NY, USA, 2010). <https://doi.org/10.1145/1755913.1755926>
144. Liu, F.T., Ting, K.M., & Zhou, Z.-H. Isolation forest. In: *2008 Eighth IEEE International Conference on Data Mining*, 413–422 (2008). <https://doi.org/10.1109/ICDM.2008.17>
145. Arnold, C., Biedebach, L., Kpfer, A., & Neunhoffer, M. The role of hyperparameters in machine learning models and how to tune them. *Political Science Research and Methods* **12**(4), 841–848 (2024) <https://doi.org/10.1017/psrm.2023.61>
146. Yang, L. & Shami, A. On hyperparameter optimization of machine learning algorithms: Theory and practice. *Neurocomputing* **415**, 295–316. <https://doi.org/10.1016/j.neucom.2020.07.061> (2020).
147. Li, Z., Kamnitsas, K. & Glocker, B. Analyzing overfitting under class imbalance in neural networks for image segmentation. *IEEE Transactions on Medical Imaging* **40**(3), 1065–1077. <https://doi.org/10.1109/TMI.2020.3046692> (2021).

148. Bichri, H., Chergui, A., & Hain, M. Investigating the impact of train/test split ratio on the performance of pre-trained models with custom datasets. *International Journal of Advanced Computer Science & Applications* **15**(2) (2024)
149. Chen, W., Yang, K., Yu, Z., Shi, Y. & Chen, C. P. A survey on imbalanced learning: latest research, applications and future directions. *Artificial Intelligence Review* **57**(6), 137. <https://doi.org/10.1007/s10462-024-10759-6> (2024).
150. Johnson, J. M. & Khoshgoftaar, T. M. Survey on deep learning with class imbalance. *Journal of big data* **6**(1), 1–54. <https://doi.org/10.1186/s40537-019-0192-5> (2019).
151. Hart, P.: The condensed nearest neighbor rule (corresp.). *IEEE transactions on information theory* **14**(3), 515–516 (1968)
152. Laurikkala, J. Improving identification of difficult small classes by balancing class distribution. In: Quaglini, S., Barahona, P., Andreassen, S. (eds.) *Artificial Intelligence in Medicine*, 63–66. (Springer, Berlin, Heidelberg, 2001). https://doi.org/10.1007/3-540-48229-6_9
153. Kubat, M., & Matwin, S. Addressing the curse of imbalanced training sets: one-sided selection. In: *ICML*, **97**, 179 (1997). Citeseer

Acknowledgements

We express our profound appreciation to Mahdi Asgharzadeh email: mahdi.azh1998@gmail.com for his significant contributions to the development of the Alarmif website.

Author contributions

Mohammad Nasirzadeh: Methodology, theoretical analysis, algorithm, implementation, experiments, and writing - Jafar Tahmoresnezhad: Supervision, architecture design, and revision - Parviz Rashidi-Khazaei: Theoretical analysis and architecture design

Funding

Not applicable.

Declarations

Consent for publication

All authors have provided their consent for the publication of this work, affirming their agreement with its content and its dissemination in the designated journal.

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to J.T.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025